URL Shortener — field notebook

- URL Shortener
- Part 1 — Setup & Architecture
  - 1. The 30,000-foot map
  - 2. Tech stack
  - 3. Repo layout
  - 4. Docker, explained from zero
  - 5. “Click to command” — what `docker compose up -d` actually does
  - 6. Fresh machine setup
  - 7. The CI/CD git workflow — and a real debugging story
- Part 2 — Backend & APIs
  - 8. Every Real HTTP API, One by One
  - 9. The Request Lifecycle, End to End
  - 10. How “Callbacks” Get a Response, and How the Response Gets Processed
  - 11. Optimization #1 — Rate Limiting
  - 12. Optimization #2 — Caching the Redirect Path
  - 13. Optimization #3 — Collision-Free ID Generation
  - 14. Optimization #4 — Decoupling Analytics from the Hot Path
  - 15. Database Schema
- Part 3 — The Frontend and the User Journey
  - 16. The component tree
  - 17. How the UI and backend actually talk
  - 18. AuthContext and ToastContext
  - 19. The full user journey
  - 20. What happens when the third party clicks the link — from the browser’s side
- Part 4 — Monitoring, CI/CD, Deployment, and Load Testing
  - 21. nginx — why a reverse proxy sits in front of the API at all
  - 22. Metrics, and what they actually tell you
  - 23. The observability story, end to end — tracing one real panel
  - 24. The CI/CD pipeline, fully
  - 25. The real deployment story
  - 26. Load testing, for real
  - 27. What’s demo vs. real, stated plainly

URL Shortener

a developer's own field notebook

Go + Fiber · React · PostgreSQL · Redis · Docker · nginx · GitHub Actions · Prometheus + Grafana

---

How to read this book. Read one chapter, close it, and explain it out loud in thirty seconds. Every major topic ends with an ★ interview tip — a question someone would actually ask, and the answer in your own words. Every fact in here is cited to a real file in this repository — nothing here is invented behavior.

Four parts, twenty-seven chapters:

1. **Setup & Architecture** (ch. 1–7) — the map, the stack, Docker explained from zero, and how to stand this whole thing up on a machine that has never seen it before.
2. **Backend & APIs** (ch. 8–15) — every endpoint, two full request traces, and the four real optimizations this system makes, each told as a problem it solves.
3. **Frontend & the User Journey** (ch. 16–20) — the component tree, how the UI and the backend actually talk, and one continuous story from a stranger's first visit to their dashboard.
4. **Monitoring & Deployment** (ch. 21–27) — nginx, metrics, the real CI/CD pipeline, and the honest story of what actually happened getting this deployed — including the parts that broke first.

Part 1 — Setup & Architecture

1. The 30,000-foot map

At its core this is a URL shortener: you give it a long URL, it hands you back a short one (`http://localhost/abc123`), and when someone visits that short link it redirects them to the original — while quietly recording that the click happened so the owner can see analytics later. Layered on top of that simple idea is everything a real production service needs: accounts (via email magic links, no passwords), API keys with rate-limit tiers, a Postgres database for durable storage, a Redis layer doing double duty as a cache and a click-event queue, a background worker draining that queue, and a full observability stack (Prometheus + Grafana) watching it all. The whole thing is one Go binary (`backend/cmd/api/main.go`) sitting behind nginx, with a React frontend built once and served as static files by that same binary — see `README.md:1–31` for the author's own one-paragraph description and diagram, which the system below mirrors with the deploy side added.



One thing worth internalizing early: the "analytics worker" in that diagram isn't a separate container or service — it's a goroutine started inside the same `main.go` process as the API server (see the CODEBASE GUIDE's "single binary … that serves the API, the built SPA, *and* runs the analytics worker goroutine in-process"). That decision shapes a lot of what follows.

> ★ interview tip **Q:** Why does a click on a short link not write directly to Postgres? **A:** The redirect path only touches Redis (cache read + a fire-and-forget push onto a Redis Stream) so the user gets redirected as fast as possible; the actual database write happens asynchronously in the analytics worker, decoupling "serve the redirect" from "record the analytics."

2. Tech stack

Pulled straight from `backend/go.mod` and `frontend/package.json`, with "why" sourced from the repo's own comments wherever they exist (CODEBASE GUIDE, Dockerfile, config.go, docker-compose files).

Layer	Dependency	Why (from the repo itself)
Backend lang/runtime	Go 1.25 (`backend/go.mod:3`)	Single static binary, cheap concurrency for the worker goroutine.
HTTP framework		

Layer	Dependency	Why (from the repo itself)
	`github.com/gofiber/fiber/v2 v2.52.14`	Used for the whole API — routing, middleware chain, static file serving (`main.go`).
ORM / DB client	`github.com/steebchen/prisma-client-go v0.47.0`	Generates a typed Go client from `backend/prisma/schema.prisma` ; CODEBASE GUIDE: “ `internal/db/prisma_store.go` — the only implementation.”
Cache/queue client	`github.com/redis/go-redis/v9 v9.6.1`	Backs four distinct concerns in `internal/redis/` — cache, counter, rate limit, stream (CODEBASE GUIDE).
Metrics	`github.com/prometheus/client_golang v1.24.1`	Exposes `/metrics` for Prometheus scraping (`main.go:79`).
Env loading	`github.com/joho/godotenv v1.5.1`	Loads `.env` for local/non-Docker runs.
Decimal math	`github.com/shopspring/decimal v1.4.0`	Available for precise numeric handling (stats aggregation).
`go.mod` `replace` directives (`golang.org/x/sys` , `x/net` , `x/text` , `x/crypto` → `github.com/golang/...` mirrors)	—	CI’s own comment (`ci.yml:36-39`) flags this as non-standard and a suspect in early CI flakiness — more in Chapter 7.
Frontend framework	React 18.2 + react-router-dom 6.22 (`frontend/package.json`)	SPA with client-side routing; CODEBASE GUIDE: “plain JS (no TypeScript).”
Build tool	Vite 5.1	Dev server on `:5173` proxying `/api` , `/health` , `/metrics` to `:8080` (CODEBASE GUIDE); `npm run build` produces `frontend/dist` , which the Go binary serves directly — not its own container.
Styling	Tailwind CSS 3.4 + `tailwind-merge` , `clsx`	Utility CSS plus helpers for conditional class merging.
HTTP client	`axios ^1.6.7`	CODEBASE GUIDE: “ `src/api/client.js` is the single axios instance (auth header + error normalization).”
Charts	`recharts ^2.12.2`	

Layer	Dependency	Why (from the repo itself)
		Renders the stats/analytics dashboards.
Animation	`framer-motion ^13.0.0`	UI transitions.
Icons	`lucide-react ^0.344.0`	Icon set for the UI.
Reverse proxy	nginx (`nginx:alpine` , `docker-compose.yml:97`)	Single public entrypoint; security headers, gzip (README.md:29-30).
Database	Postgres 16 (`postgres:16-alpine`)	Primary datastore, Prisma-managed schema.
Cache/queue	Redis 7 (`redis:7-alpine`)	Password-protected (`--requirepass`), used for cache + rate limiting + the click-event stream.
Mail sink (dev)	MailHog	SMTP catcher for magic-link/ verification email in local dev, see Chapter 4.
Metrics stack	Prometheus + Grafana	Scrapes `/metrics` every 15s; dashboards provisioned automatically (README.md:93-99).

> ★ interview tip **Q:** Why does the Go backend serve the frontend's built files instead of running a separate frontend container? **A:** CODEBASE GUIDE is explicit: "Built output is served by the Go binary, not by a separate container" — one binary, one container, one thing to deploy and health-check; the Dockerfile's frontend-builder stage exists only to produce static files that get copied into the final image (Chapter 4).

3. Repo layout

Verified against the actual tree (`ls -la` from repo root), not memory:

- `backend/` — the Go service: `cmd/api` (entrypoint, `main.go`), `internal/{auth,config,db,handlers,mail,metrics,middleware,redis,shortcode,store,worker}` , `prisma/` (schema.prisma, the source of truth for the DB shape).
- `frontend/` — React/Vite app: `src/{api,components,context,hooks,pages,utils}` , `public/` , `dist/` (build output — this is what gets copied into the Docker image), `node_modules/` (local install, not shipped).
- `docs/system-design/` — `01` through `11` numbered design docs plus `adr/` (architecture decision records) — CODEBASE GUIDE calls these out explicitly as recording *current and proposed* state, with ADRs for analytics decoupling, the authorization model, nginx, and CI/CD.

- `.github/workflows/` — `ci.yml` (test + build + push to GHCR) and `deploy.yml` (self-hosted-runner deploy, triggered by `workflow_run` off CI).
- `.devcontainer/` — GitHub Codespaces config (Docker-in-Docker, auto-bring-up for demos — README.md:60-83).
- `nginx/` — `nginx.conf`, the reverse-proxy config referenced by both compose files.
- `monitoring/` — `prometheus/alerts.yml` and `grafana/provisioning/...` (dashboards + datasource, auto-provisioned).
- `load-tests/` — k6 load test scripts and results (the subject of the most recent commit on this branch, `c6944f7`).
- `scripts/` — demo/seed data tooling (`seed-demo-data.sql`), explicitly *not* a production seeding tool (README.md:101-108).
- `notes/` — this file.
- Root-level docs: `README.md`, `GETTING_STARTED.md`, `DEPLOY.md`, `CASE_STUDY.md`, `SYSTEM_DESIGN_TOPICS.md`, `CLAUDE.md`, `interview.md`.
- Root-level infra: `docker-compose.yml` (dev stack), `docker-compose.prod.yml` (production overlay), `prometheus.yml` (scrape config), `.env` / `.env.example`.

> ★ interview tip **Q:** Where does the database schema actually live, and what's generated vs. hand-written? **A:** `backend/prisma/schema.prisma` is hand-written and the source of truth; `backend/internal/db/*_gen.go` is generated from it by `prisma-client-go generate`, is gitignored, and must be regenerated after any schema edit (CODEBASE GUIDE, "Commands" section).

4. Docker, explained from zero

Before getting into *this* repo's Docker setup, it's worth being concrete about the problem it solves. Without Docker, running this project means: install Go 1.25 exactly, install Node 20 exactly, install Postgres 16 and get a user/database created with the right credentials, install Redis 7, make sure none of their default ports collide with something else already on your machine, and get all four talking to each other with the right connection strings — and then do that identically on every teammate's machine and on the deploy target. One mismatched version (say Postgres 15 instead of 16, or a different Prisma engine binary for your OS) and something subtly breaks.
`GETTING_STARTED.md`'s "Option A" (no-Docker path) spells out exactly this manual sequence — `brew install go node postgresql@16 redis`, manually creating the Postgres user and database, manually running three separate `prisma-client-go` commands — and that's the *easy* case where every tool installs cleanly. Docker's answer: package each dependency as an image (a frozen, versioned filesystem + startup command) and let `docker compose` start all of them together, networked, with the exact versions pinned in the compose file. README.md states this plainly: "Nothing else needs to be installed on the host — Go, Node, Postgres, Redis and Prisma all run inside containers" (`README.md:35-37`).

Walking `backend/Dockerfile` stage by stage:

1. **`frontend-builder` stage** (`backend/Dockerfile:1–11`, `FROM node:20-alpine`): copies `frontend/package*.json` first (so `npm install` is cached and only reruns when dependencies change, not on every source edit), then copies the rest of the frontend source and runs `npm run build`. This stage's only job is to produce `frontend/dist` — a folder of static HTML/JS/CSS.
2. **`builder` stage** (`backend/Dockerfile:14–33`, `FROM golang:1.25-alpine`): installs `openssl` because, per the Dockerfile's own comment, "Alpine's musl libc needs libc6-compat and openssl for Prisma's engine binaries" (line 17). Copies `go.mod` / `go.sum` first for the same caching reason as the frontend stage, then the rest of the backend source. Then — critically — it runs `prisma-client-go prefetch` and `generate` **inside this container**, not on the developer's own machine. The Dockerfile's comment explains exactly why (lines 25-29): "generate the Prisma client HERE … This ensures the query-engine binary that gets embedded matches the container's actual platform (Alpine/musl), not whatever OS you're developing on (e.g. macOS). This was the cause of the 'ensure: no binary found' runtime error." In other words: Prisma's client embeds a compiled, platform-specific binary, and generating it on a Mac bakes in a macOS binary that then can't run inside the Linux Alpine container at runtime — generating it inside the same container that will run it sidesteps the mismatch entirely. Then it builds the actual Go binary with `CGO_ENABLED=0 GOOS=linux`.
3. **Run stage** (`backend/Dockerfile:36–54`, `FROM alpine:3.19`): a minimal final image — just `ca-certificates`, `openssl`, `wget`, a non-root `appuser`. Copies *only* the compiled binary from `builder` and the `dist/` folder from `frontend-builder` — none of the Go toolchain, Node toolchain, or source code ends up in the final image, keeping it small and reducing attack surface. It declares a `HEALTHCHECK` hitting `/health` and runs as the non-root `appuser`.

Why build the frontend in a *separate* stage rather than, say, running `npm run build` as a step inside the Go stage? Because the final image needs the `dist/` output but never needs Node itself at runtime — multi-stage builds let you use whatever tooling you need to *produce* an artifact without shipping that tooling in the image that actually runs in production.

Now `docker-compose.yml`, service by service (`docker-compose.yml:1–146`):

- `postgres` — `postgres:16-alpine`, exposes `5432` to the host (dev convenience — you can `psql` into it directly), has a `pg_isready` healthcheck other services wait on.
- `redis` — `redis:7-alpine`, `--requirepass` from `.env`, `--appendonly yes` for persistence, its own healthcheck via `redis-cli ping`.
- `mailhog` — SMTP sink + web UI, exposed on `1025` / `8025` in dev so you can read verification emails in a browser.
- `migrate` — a one-shot container (`restart: "no"`) that runs `prisma-client-go db push` against Postgres once, then exits; `api` depends on it completing successfully (`condition: service_completed_successfully`), so the schema always exists before the API starts.
- `api` — builds from `backend/Dockerfile`, loads `.env` wholesale via `env_file`, then *overrides* a handful of values (`DATABASE_URL`, `REDIS_ADDR`, etc.) because, as the compose file's own comment explains (`docker-compose.yml:68–70`): "inside Docker's network,

services are reachable by their service name, not localhost. `.env` itself stays correct for running `go run ./cmd/api` directly on your host." This is the Docker-networking rule from CLAUDE.md §4 in action — containers talk to each other as `postgres:5432`, `redis:6379`, never `localhost`.

- `nginx` — the single public entrypoint on `:80`, reverse-proxying to `api`.
- `prometheus` / `grafana` — scrape and visualize `/metrics`.

`docker-compose.prod.yml` is a *second, separate file* layered on top with `-f docker-compose.yml -f docker-compose.prod.yml`, never merged into the first. Its own header comment explains why a separate overlay rather than one file with conditionals: dev stays exactly as-is when you just run `docker compose up`, and production gets a deliberately different, stricter posture without any risk of dev and prod configs leaking into each other. Concretely it changes five things (`docker-compose.prod.yml:10–55`): strips host port publishing from everything except nginx (so Postgres/Redis/Prometheus/Grafana/the API are reachable only over the internal Docker network, never from the internet); replaces the hardcoded Postgres password with a required `${POSTGRES_PASSWORD:?...}` variable that hard-fails the deploy if unset rather than silently using a default; keeps MailHog running but *unpublished* (reachable only via a deliberate SSH tunnel, since its web UI lists every user's verification link); disables the automatic `migrate db push` in favor of a no-op, because `db push` has no migration history or rollback path and could silently corrupt production data on a schema change; and swaps `build:` for a pinned `image:` from GHCR so production always runs exactly what CI tested.

> ★ interview tip **Q:** Why generate the Prisma client inside the Docker build instead of once on a developer's machine and committing the result? **A:** The generated client embeds a platform-specific query-engine binary; generating it on macOS and shipping it into a Linux/Alpine container produces a binary mismatch ("ensure: no binary found" at runtime) — generating it inside the same Alpine builder stage guarantees the binary matches where it will actually run (`backend/Dockerfile:25–29`).

> ★ interview tip **Q:** Why does `docker-compose.prod.yml` exist as a separate overlay file instead of one compose file with environment-based conditionals? **A:** So `docker compose up` (plain dev) is completely unaffected by production-only hardening — the overlay only takes effect when explicitly layered with `-f docker-compose.yml -f docker-compose.prod.yml`, keeping the blast radius of a prod-specific change away from local dev (`docker-compose.prod.yml:1–9`).

5. "Click to command" — what `docker compose up -d` actually does

First, a distinction that trips people up: an **image** is a frozen, read-only template — a filesystem snapshot plus metadata about what to run (e.g. `postgres:16-alpine` or the image built from `backend/Dockerfile`). A **container** is a running instance of that image — a process (or group of processes) with its own isolated filesystem view, network namespace, and process tree, layered on top

of the image's filesystem. You can start many containers from the same image; stopping a container doesn't delete the image.

When you run `docker compose up -d`, roughly this happens: Compose reads `docker-compose.yml`, resolves each service's `build` context or `image` reference (here, `api` has a `build: {context: ., dockerfile: backend/Dockerfile}`, everything else has a plain `image:`). For anything with a `build:`, it builds the image (running the Dockerfile stages from Chapter 4) if it isn't already built or if inputs changed; for plain `image:` services it pulls them from Docker Hub if not already cached locally. It creates a default bridge network so containers can resolve each other by service name. Then it starts containers — and this is where `depends_on` with `condition:` matters concretely in this repo: `postgres` and `redis` each have a `healthcheck` (`pg_isready`, `redis-cli ping`), so `migrate`'s `depends_on: postgres: condition: service_healthy` means it won't even attempt to start until Postgres is actually accepting connections, not just "the container process exists." `migrate` then runs its one-shot `db push` and exits 0; `api`'s `depends_on: migrate: condition: service_completed_successfully` means the API container is held back until that schema push has *finished successfully* — so the API never races against a half-applied schema. `api` also waits on `postgres` / `redis` being healthy and `mailhog` merely being started. Only once all of that is satisfied does the `api` container's process actually start, and *its own* healthcheck (hitting `/health`) is what `nginx`'s implicit dependency and any external orchestration would key off next.

> ★ interview tip **Q:** What's the practical difference between `depends_on: condition: service_started` and `service_healthy` in this compose file? **A:** `service_started` just means the container process has been launched (e.g. `mailhog`); `service_healthy` means the service's own `healthcheck` command is passing (e.g. Postgres actually accepting connections) — `migrate` needs the latter for `postgres`, otherwise it could try to connect before Postgres has finished initializing.

6. Fresh machine setup

For someone who has never touched this project, the exact sequence (per `README.md:39-43` and `GETTING_STARTED.md`'s Option B, which it calls "recommended, easiest"):

```
git clone <repo-url> && cd Url-Shortner
cp .env.example .env              # fill in anything marked change-me
cp backend/.env.example backend/.env
docker compose up --build
```

That's it — no local Go, Node, Postgres, or Redis installs needed (`GETTING_STARTED.md:110-111`). The `migrate` service applies the Prisma schema automatically before `api` starts, so there's no manual `db push` step. What to expect at each URL once containers are up (`README.md:45-53`, `GETTING_STARTED.md:122-130`):

URL	What's there
`http://localhost:80`	The app itself, via nginx
`http://localhost:8080`	The API directly (bypassing nginx)
`http://localhost:3000`	Grafana (`admin` / whatever `GF_SECURITY_ADMIN_PASSWORD` is set to)
`http://localhost:9090`	Prometheus
`http://localhost:8025`	MailHog's web UI, for reading verification/magic-link emails

To stop: `docker compose down` keeps the Postgres/Redis volumes; `docker compose down -v` destroys them — per CLAUDE.md §22 this needs explicit approval before ever running it on anything that matters.

For the production-hardened version: that's a different, deliberately heavier path, condensed from `DEPLOY.md`. It uses a **self-hosted GitHub Actions runner** on the target machine (Windows+WSL2 in the documented case) rather than GitHub's own cloud runners, because the deploy step needs to run `docker compose` *on that specific machine*. One-time setup: install Docker Desktop with WSL2 integration, clone the repo inside WSL, register the runner via **Settings → Actions → Runners → New self-hosted runner**, then install it as a systemd service (`sudo ./svc.sh install && start`) so it survives reboots (`DEPLOY.md:33–59`). Seven GitHub Actions Secrets are required before any deploy will succeed — `POSTGRES_PASSWORD` , `REDIS_PASSWORD` , `IP_HASH_SECRET` , `BASE_URL` , `CORS_ALLOWED_ORIGINS` , `GF_SECURITY_ADMIN_PASSWORD` , `MAIL_FROM` (`DEPLOY.md:67–81`) — `deploy.yml` checks each is non-empty before writing `.env` , and `docker-compose.prod.yml` re-enforces several via `${VAR:?...}` so a secret removed later is still caught. The actual deploy (`git push origin main` , or a manual `workflow_dispatch`) writes `.env` from those secrets, runs `docker compose -f docker-compose.yml -f docker-compose.prod.yml pull` to fetch the image `ci.yml` just built, brings the stack up with `up -d --remove-orphans` *using the prod overlay* (never the bare file), polls until nothing is `starting` / `unhealthy` , then curls `http://localhost/health` through nginx and fails the job if the app never answers (`DEPLOY.md:122–138`). To reach it from outside that machine without opening router ports, the documented approach is a free, outbound-only **Cloudflare Tunnel** (`DEPLOY.md:160–167`), which is also why it appears in the Chapter 1 diagram's deploy side.

> ★ interview tip **Q:** Why is a self-hosted runner required for `deploy.yml` specifically, when `ci.yml` runs fine on GitHub's own cloud runners? **A:** The deploy job has to run `docker compose up` *on the actual target machine* to bring the live stack up there — a GitHub-hosted cloud runner has no access to that machine's filesystem, network, or running containers, so the job must execute on a runner installed on the machine itself.

7. The CI/CD git workflow — and a real debugging story

The intended flow is standard: push a branch → open a PR → `ci.yml` runs automatically on both `push` and `pull_request` to `main` → once merged to `main`, `ci.yml` runs again, and on success, `deploy.yml` fires via a `workflow_run` trigger (not a plain `push` trigger) specifically so the deploy waits for CI's Docker build+push to finish before trying to pull an image that might not exist yet (`deploy.yml`'s own header comment explains the race this avoids: both workflows fire on the same push, but the deploy job — "a checkout and a file write" — reaches `docker compose pull` far faster than a full Node+Go+Prisma multi-stage build finishes).

The real story, visible in `git log --oneline --all` and in the comments left behind in `ci.yml` itself: CI failed on **every single run from the first commit**. The commit history shows the actual sequence of hypotheses:

```
ec1a9da fix: disable Go module caching in CI, add verbose test output
62e9c98 Merge pull request #4 from secretJod/fix/ci-disable-go-cache
...
0ed94c7 fix: generate the Prisma client in CI before running tests
ee077df Merge pull request #5 from secretJod/fix/ci-generate-prisma-client
6e67a8f fix: publish multi-arch (amd64+arm64) Docker images from CI
51bc89f Merge pull request #6 from secretJod/fix/ci-multiarch-image
```

The **first wrong hypothesis** (branch `fix/ci-disable-go-cache`, commit `ec1a9da`) was that GitHub's Go module cache itself was stale or corrupted — plausible on paper, since `backend/go.mod` has non-standard `replace` directives pointing `golang.org/x/sys` / `x/net` / `x/text` / `x/crypto` at `github.com/golang/...` mirror repos instead of the normal module proxy, which is exactly the kind of setup that can produce a bad cached entry. `ci.yml`'s own comment on the `cache: false` line documents this reasoning *and* the fact that it didn't actually fix anything — it was a scoped, logged elimination of one hypothesis, with the comment explicitly noting: "If disabling this does NOT fix it, that rules this out and the actual failure text from the Actions log is needed next" (`ci.yml:31-43`).

The **real root cause**, found on the next branch (`fix/ci-generate-prisma-client`, commit `0ed94c7`): `backend/internal/db/prisma_store.go` references types (`PrismaClient`, `NewClient`, `Link`, `User`, …) that only exist in Prisma's *generated* output, and `internal/db`'s own `.gitignore` excludes `*_gen.go` — so on a fresh CI checkout, those files simply don't exist yet. Without a generate step, `internal/db` (and everything importing it, including `cmd/api`) **fails to compile**, not just fails a test. `ci.yml`'s comment nails exactly why this was easy to misdiagnose as a flaky test: "`go test` across the whole module still reports every OTHER package's real test results before failing overall, which is what made this easy to misread as a flaky/environment-specific test failure rather than two packages refusing to build" (`ci.yml:51-62`). The fix was two lines — `prisma-client-go prefetch` then `generate`, both run before `go test ./... -v` (`ci.yml:68-70`), mirroring exactly what the Dockerfile already did for the same reason (Chapter 4).

A third, unrelated fix landed right after (`fix/ci-multiarch-image`, `6e67a8f`): the Docker image CI builds and pushes was amd64-only, which would fail to pull on an arm64 deploy target (e.g. Apple

Silicon); `docker/setup-qemu-action` plus building with `platforms: linux/amd64,linux/arm64` fixed that, documented in `ci.yml:92-100` and `:135-140`.

> ★ interview tip **Q:** Why did disabling Go's module cache not fix the CI failures, even though the `replace` directives made it a reasonable first suspect? **A:** The actual failure wasn't a caching problem at all — it was that `internal/db`'s generated Prisma client files are gitignored and never existed on a fresh CI checkout, so two packages failed to *compile*; no amount of cache invalidation changes whether generated source files are present on disk.

> ★ interview tip **Q:** Why did the real compile failure look like a flaky test rather than an obvious build error? **A:** `go test ./...` runs and reports every package independently — packages unaffected by the missing generated code still ran and passed their real tests, so the output showed mostly-green results with the two broken packages' compile errors buried among them, rather than one loud "build failed" message.

Part 2 — Backend & APIs

8. Every Real HTTP API, One by One

Everything the server exposes is registered in one place: `backend/cmd/api/main.go`, lines 72–144. There's no separate router file — Fiber routes are wired directly in `main()`. Route order matters a lot here (see the comment at line 136), because `/:shortCode` (line 134) is a catch-all: anything registered after it that isn't the final 404 handler would silently become a short-code lookup instead. That's why every `/api/*` route, every SPA route, and the static file server all sit above it, and a bounded `app.Use(...)` 404 handler sits below it (lines 142–144) to keep the Prometheus `path` label from exploding on scanner traffic.

Method	Path	Middleware chain (in order)	Handler	Auth
GET	`/health`	—	`health.Check`	none
GET	`/metrics`	—	`promhttp.Handler()` (adapted)	none
GET	`/api/metrics`	—	inline closure → `appMetrics.Snapshot()`	none
POST	`/api/keys`	`rateLimitMW`	`apiKeys.CreateKey`	none (this *creates* the identity)
GET	`/api/keys/verify`	`rateLimitMW`	`apiKeys.VerifyKey`	

Method	Path	Middleware chain (in order)	Handler	Auth
				token in query string
POST	`/api/shorten`	`authMW` (optional) → `rateLimitMW`	`shorten.Shorten`	optional Bearer key
GET	`/api/stats/top`	`rateLimitMW`	`stats.GetTopLinks`	none (public, aggregate-only)
GET	`/api/stats/:shortCode`	`requireAuthMW` → `rateLimitMW`	`stats.GetLinkStats`	required + ownership
GET	`/api/links`	`authMW` (optional)	`stats.GetUserLinks`	required in practice (401 inside handler if missing)
GET	`/api/links/:shortCode/clicks`	`requireAuthMW` → `rateLimitMW`	`stats.GetRecentClicks`	required + ownership
DELETE	`/api/links/:shortCode`	`requireAuthMW` → `rateLimitMW`	`stats.DeleteLink`	required + ownership
GET	`/`, `/login`, `/dashboard`, `/stats/*`, `/top`, `/admin`	—	`serveIndex` (SPA)	none
GET	`/:shortCode`	`rateLimitMW`	`redirect.Redirect`	none (catch-all)
any	`/*` (after everything above)	—	inline 404 closure	—

Notice the asymmetry in middleware ordering: `/api/shorten` runs `authMW` *before* `rateLimitMW` (main.go:98), and so does every stats route. That ordering isn't cosmetic — `middleware.RateLimit` (ratelimit.go:40) reads `GetAPIKey(c)` from `c.Locals`, which only exists if an auth middleware ran first and populated it. Register them the other way around and every authenticated request would silently fall back to the much stricter anonymous IP bucket. This is called out explicitly in the project's own internal/middleware doc comment (middleware/auth.go:10–15).

Deeper look: `POST /api/shorten`

Request: `{"url": "...", "custom_alias": "optional", "expires_at": "optional RFC3339"}` (shorten.go:30–34). Response on success is `201 Created` with `{"short_url", "short_code", "long_url"}` (shorten.go:36–40, 96–100).

The handler (shorten.go:42–101) does, in order: parse body (400 on bad JSON) → validate the URL is a real `http` / `https` URL via `url.ParseRequestURI` (shorten.go:127–136, rejecting things like `javascript:` or bare strings — this is the open-redirect/dangerous-protocol defense) → parse `expires_at` if present (400 on bad format) → resolve a short code, either reserving the custom alias or minting one from the Redis counter (shorten.go:105–125, see chapter 13) → attach `UserID` from the authenticated API key if present (shorten.go:82–84, so anonymous shortens stay ownerless) → `CreateLink` in Postgres (shorten.go:86) → **write-through cache** into Redis immediately (shorten.go:94) so the very first redirect is already a cache hit instead of a guaranteed miss. If the Postgres insert fails after a custom alias was reserved, the alias reservation is released (shorten.go:87–90) so it doesn't get stuck locked for 30 seconds for nothing.

★ **interview tip** — "Why release the alias reservation on a failed insert instead of just letting the 30-second TTL expire it?" Because a client retrying immediately (a very normal thing for an HTTP client to do on a 500) would get a spurious 409 "already taken" for up to 30 seconds, on an alias nobody actually holds. Releasing it makes the retry path correct, not just eventually-correct.

Deeper look: `GET /:shortCode` (the redirect)

No request body. Two outcomes: `302 Found` with a `Location` header on success, or JSON `404` / `500` on failure (redirect.go:38, 51, 54, 64). This is covered in full line-by-line detail in chapter 9's second trace — the short version is cache-first (redirect.go:30), Postgres on miss with a cache backfill (redirect.go:47–58), and a fire-and-forget click event pushed onto a Redis Stream either way (redirect.go:35, 61).

Deeper look: the magic-link pair — `POST /api/keys` + `GET /api/keys/verify`

This project has no passwords. Identity is: email → magic link → API key. `POST /api/keys` (apikeys.go:63–105) takes `{"email": "..."}`, validates it with `net/mail.ParseAddress` (apikeys.go:69), and calls `GetOrCreateUserByEmail` (apikeys.go:75), which is itself implemented as a Prisma `Upsert` rather than find-then-create specifically to avoid a TOCTOU race on the `email` unique constraint when two requests for the same brand-new address land concurrently (prisma_store.go:144–149, 150–160). It then generates a random 32-byte token, stores only its SHA-256 hash (apikeys.go:80–92, same "never persist the raw secret" pattern used for API keys), and emails a link containing the *raw* token (apikeys.go:94–95). The response is `202 Accepted` with no key in it — the key doesn't exist yet.

`GET /api/keys/verify?token=...` (apikeys.go:110–159) is what actually issues the key: it hashes the incoming token, looks up the `VerificationToken` row by that hash, checks expiry (1 hour — see the long comment at apikeys.go:19–32 explaining *why* 1 hour instead of the more obviously-secure 15 minutes: this deployment relays mail through MailHog manually, so a human sits in the loop between "email sent" and "link clicked"), deletes the token immediately so it can't be replayed (apikeys.go:133, *before* even marking the user verified — consume-first ordering), marks the user verified, and only then calls `auth.GenerateAPIKey()` (apikeys.go:141) which returns the raw key once and its hash for storage. The raw key is returned in the `201` response body and never persisted

anywhere (apikeys.go:155–158) — if the user loses it, there is no recovery path, only "request a new verification email and get a new key."

★ **interview tip** — "What stops someone from hitting `/api/keys/verify` twice with the same token and getting two API keys?" The token row is deleted (apikeys.go:133) before the key is issued, and the lookup (apikeys.go:119) returns `store.ErrNotFound` for a deleted token — so the second request gets a clean 400 "invalid or already-used token," not a second key.

---

9. The Request Lifecycle, End to End

Trace 1: `POST /api/shorten`

1. **nginx** receives the request on port 80 and proxies it to `api:8080` over the Docker network (service name, not `localhost` — see §4/CLAUDE.md and the service map in the codebase guide).
2. Fiber's middleware chain runs in registration order (main.go:54–70):
   - `recover.New()` — wraps everything downstream in a panic recoverer so a bug in a handler returns a 500 instead of killing the process.
   - `logger.New()` — structured access logging.
   - `cors.New(...)` — validates `Origin` against `CORS_ALLOWED_ORIGINS` (env-driven, main.go:59–63) and sets the CORS headers.
   - `middleware.MetricsMiddleware(appMetrics)` — legacy atomic-counter metrics (main.go:67).
   - `middleware.PrometheusMetricsMiddleware()` — records the Prometheus HTTP histograms/counters with the route's path label (main.go:70).
3. The route-specific chain then runs: `authMW` (`OptionalAPIKeyAuth` , auth.go:22–48) reads `Authorization` , and if present, extracts the bearer token, hashes it, and looks it up via `GetAPIKeyByHash` (one Postgres query) — on success it stashes the `*store.ApiKey` in `c.Locals(LocalsAPIKey)` . No header at all just calls `c.Next()` as anonymous. A header that's *present but invalid* is rejected outright with 401 (auth.go:30–34, 39–43) — deliberately not silently downgraded to anonymous.
4. `rateLimitMW` (`RateLimit` , ratelimit.go) reads the API key that step 3 may have put in locals, picks a bucket+limit (chapter 11), and calls `rdb.Allow(...)` — one Redis round trip (a `TxPipeline` with 4 commands). 429 with `Retry-After` on rejection, otherwise `c.Next()` .
5. `shorten.Shorten` (shorten.go:42) runs: parses and validates the body, then **one of two possible Redis calls** — either `ReserveAlias` (`SETNX`) for a custom alias, or `NextID` (`INCR`) for an auto code — then **one Postgres write** (`CreateLink` , an `INSERT`), then **one more Redis write** (`SetLongURL` , the write-through cache). So the order on the wire is: Redis (alias reserve or counter incr) → Postgres (insert) → Redis (cache write). If the Postgres insert fails and a custom alias was reserved, there's a cleanup Redis call (`ReleaseAlias`) after it.

6. The handler returns the `shortenResponse` JSON directly — Fiber's `c.JSON(...)` writes to the underlying `fasthttp` response and returns; there's no separate serialization step the way an Express callback would need.
7. nginx passes the response straight back to the client unmodified.

Trace 2: `GET /:shortCode` — cache hit

1. nginx → `api:8080`, same middleware stack as above minus the auth middleware (this route doesn't register one — redirects are always anonymous as far as auth goes) — just `rateLimitMW` (main.go:134).
2. `redirect.Redirect` (redirect.go:25) calls `h.Redis.GetLongURL(ctx, shortCode)` (cache.go:47) — one Redis `GET`. It comes back with a JSON-decoded `{url, id}` (cache.go:56–60).
3. `fireClickEvent` (redirect.go:71–80) builds a `ClickEvent` struct and calls `PushClickEvent`, which is one Redis `XADD` (stream.go:53–61). This happens *before* the redirect response is sent, but errors from it are explicitly discarded (redirect.go:79) — a Redis Stream outage never fails a redirect.
4. `c.Redirect(longURL, fiber.StatusFound)` (redirect.go:38) — `302`, done. **Zero Postgres round trips on this path.**

Trace 2b: `GET /:shortCode` — cache miss

1–2. Same as above, but `GetLongURL` returns `redis.ErrCacheMiss` (cache.go:49–50, 14). 3. Falls through to `h.Store.GetLinkByShortCode(ctx, shortCode)` (redirect.go:47) — one Postgres `SELECT ... WHERE short_code = $1` (prisma_store.go:76–79, backed by the unique index on `short_code`). If not found (or expired — prisma_store.go:106–109 treats an expired link as not-found), returns `404` immediately with no cache write. 4. On success: `SetLongURL` backfills the cache (redirect.go:58) — one Redis `SET` with a TTL — *before* firing the click event, so step 5 can't somehow race ahead of the cache being warm. 5. `fireClickEvent` — one Redis `XADD`, same as the hit path. 6. `302` redirect.

So a cache miss costs exactly one Postgres read and two Redis writes (cache backfill + stream push) instead of the hit path's one Redis read and one Redis write.

---

10. How "Callbacks" Get a Response, and How the Response Gets Processed

This is really two different programming models stitched together by one HTTP response, and conflating them is a common source of confusion.

Backend (Go/Fiber) — not a callback model at all. A Fiber handler is a plain function, `func(c *fiber.Ctx) error` (every handler in this codebase has that exact signature — e.g. `func (h *ShortenHandler) Shorten(c *fiber.Ctx) error` at shorten.go:42). There's no "send a response,

then later call back" — the function *is* the entire request lifecycle. Inside it, `c.JSON(...)`, `c.Redirect(...)`, `c.Status(...).JSON(...)` etc. write bytes directly onto the underlying `fasthttp.RequestCtx`'s response buffer, synchronously, in whatever order the code calls them. The function then `return`s (usually `return c.JSON(...)`, returning that write call's own error value so Fiber's error-handling middleware can see if the write itself failed). Once the handler returns, Fiber flushes the buffered response to the socket. There is no event loop handing control back to a callback later — the goroutine that's running this handler *is* the thing serving the socket, for the duration of the request.

Frontend (React/axios) — this is where the actual async callback model lives. `frontend/src/api/client.js` wraps `axios.create(...)` with two interceptors (client.js:10–46): a request interceptor that reads the API key out of `localStorage` and attaches `Authorization: Bearer <key>` (client.js:12–14), and a response interceptor that normalizes errors — a `401` clears the stored key and redirects to `/login` (client.js:27–32), a `429` gets rewritten to a friendly "too many requests" message (client.js:36–37), and anything else gets shaped into `{status, message, data}` and rejected (client.js:42). Every call site in the React app calls `apiClient.post(...)` / `apiClient.get(...)`, which returns a `Promise`. Whatever component made the call either `await`s it inside an `async` function or chains `.then()/.catch()` — and it's *that* resolution, not anything on the Go side, that actually updates React state and re-renders the UI.

Bridging the two halves: by the time axios's promise resolves on the frontend, the Go handler that produced the response has already returned and its goroutine is gone — the "connection" between them is purely the HTTP response bytes that traveled over the wire (through nginx) in between. The backend's synchronous write-and-return model and the frontend's asynchronous promise-resolution model never share any runtime state; the HTTP response is the entire contract.

★ **interview tip** — "If a Fiber handler panics halfway through writing a partial response, what does the client see?" `recover.New()` (main.go:54) catches the panic before it reaches the OS-level connection handler and converts it into a `500`, but only if the panic happens *before* Fiber has already started flushing bytes — once the status line and headers are written, you can't retroactively rewrite the status code. In practice, because every handler here builds the full response fiber.Map before calling `c.JSON` / `c.Redirect` once, there's no "half-written" response case in this codebase.

---

11. Optimization #1 — Rate Limiting

The problem: with no limiter, one script hammering `/api/shorten` or scraping sequential short codes off `/:shortCode` can monopolize the process, degrade latency for everyone else, and (for the redirect path) do cheap enumeration — so a limiter sits in front of effectively every route that touches a store (`rateLimitMW` on `/api/keys`, `/api/keys/verify`, `/api/shorten`, every `/api/stats/*` and `/api/links*` route, and `/:shortCode` — see the table in chapter 8).

The exact algorithm — sliding-window log, in `backend/internal/redis/ratelimit.go:19–55`:

```
ZREMRANGEBYSCORE key 0 <now - window>   // drop entries older than the window
ZCARD key                               // count what's left
ZADD key <now_ns> <member>              // record this request optimistically
EXPIRE key <window>                     // so idle keys don't live forever
```

All four run in a single `TxPipeline` (ratelimit.go:24–30) — one Redis round trip, not four. The count returned by `ZCARD` is read *before* the `ZADD` landed logically (Redis pipelines execute commands in order but the client reads `countCmd.Val()` after `Exec`, so it reflects the count prior to this request's own entry — ratelimit.go:34). If that count is already `>= limit`, the request is over the limit; the member that was optimistically added gets removed again with `ZREM` (ratelimit.go:38) so a rejected request doesn't itself count against the window. `retryAfter` is then estimated from the oldest surviving entry's score (ratelimit.go:41–50).

Each sorted-set member is `"<unix_nanos>-<4 random hex bytes>"` (ratelimit.go:22, util.go:17–21) — the random suffix exists purely to avoid two requests landing on the exact same nanosecond and colliding as the *same* sorted-set member (which would silently undercount, since a set member is unique).

Why sliding-window-log beats fixed-window: a fixed window (e.g. "60 requests per calendar minute") lets a client burst 60 requests at 0:59 and another 60 at 1:01 — 120 requests in 2 seconds, still "compliant." The sliding log has no window boundary to burst across: it always asks "how many requests happened in the last 60 seconds, as of right now," so a client literally cannot get more than `limit` requests into *any* 60-second span. The cost is `O(log N)` per request for the sorted-set ops instead of `O(1)` for a fixed counter — a reasonable trade at the request volumes a per-client limiter itself has to handle (tens to hundreds of requests per minute, not millions).

The three tiers (middleware/ratelimit.go:16–27):

Tier	Limit	Window	Who
anonymous	20	1 minute	per-IP, no API key
`standard`	60	1 minute	default tier for new keys
`pro`	600	1 minute	elevated tier

The bucket key is `ratelimit:apikey:<id>` when authenticated or `ratelimit:ip:<ip>` when anonymous (middleware/ratelimit.go:45, 48) — so an authenticated user's limit follows their key across IPs/devices, while an anonymous caller is bucketed purely by IP.

What happens when Redis is down: `rdb.Allow(...)` returns an error, and the middleware's response to that is `return c.Next()` (middleware/ratelimit.go:53–57) — it **fails open**. The comment right there calls this out explicitly: "a Redis hiccup shouldn't take down the whole API." This is the right call for this system specifically because the alternative (fail closed, reject everything) would turn a transient cache outage into a full API outage for every legitimate user, in exchange for blocking abuse that Redis being down already makes harder to sustain at scale anyway (an attacker

hammering a degraded backend has bigger problems than the rate limiter). The trade-off being accepted is: during a Redis outage, abuse protection is temporarily off.

★ **interview tip** — "Why remove the optimistically-added member on rejection instead of just not adding it until the count check passes?" Because the count check and the add happen in the *same* pipelined round-trip for performance — splitting it into "count, then decide, then maybe add" would mean two round trips and a race window between them where two concurrent requests could both see a count just under the limit and both get admitted, overshooting it.

---

12. Optimization #2 — Caching the Redirect Path

The problem: a redirect is the single highest-volume endpoint in a URL shortener by a wide margin — every click on every link ever created hits it. If every one of those went straight to Postgres, the database becomes the bottleneck the moment traffic is more than "a few requests a second," and a transient DB slowdown would directly translate into slow redirects for every user clicking any link at that moment.

Cache-aside, implemented in `backend/internal/redis/cache.go`: the redirect handler checks Redis first (`GetLongURL`, cache.go:47); on a miss it reads Postgres and then writes the result back into Redis (`SetLongURL`, cache.go:70) so the *next* request for that code is a hit. Redis is never the system of record — Postgres is — so Redis can be flushed entirely and the system degrades to "always a cache miss," not "data loss."

Exact TTL logic (cache.go:70–85): default TTL is `defaultCacheTTL = 24 * time.Hour` (cache.go:29). If the link has an explicit `expiresAt`, the cache entry's TTL is capped to `min(time.Until(expiresAt), 24h)` (cache.go:72–75) — so a link expiring in 10 minutes gets cached for 10 minutes, not 24 hours (which would let a redirect serve a link past its intended expiry), while a link with no expiry or a far-future one still gets periodically refreshed from Postgres rather than living in cache forever.

Why the cached value is `{url, id}`, not just the bare URL string (cache.go:31–38): the redirect handler needs the link's database ID to attach to the click event it pushes onto the analytics stream (redirect.go:34–36). Without the ID cached alongside the URL, a cache *hit* would still need a Postgres lookup just to get the ID for analytics — which would defeat half the point of caching. The struct is JSON-encoded (`cachedLink`, cache.go:35–38) and there's explicit backward-compatibility handling for the pre-this-feature plain-string cache format: if JSON decoding fails or comes back with an empty URL, the raw string is treated as a bare URL with `linkID=0` (cache.go:56–63), and the redirect handler skips firing a click event when `linkID` is `0` (redirect.go:33–34) rather than writing a garbage/zero `link_id`.

Invalidation on delete: `DeleteLink` (stats.go:191–223) deletes the row from Postgres first, then calls `h.Redis.InvalidateLongURL(...)` (stats.go:218–219), which is a plain Redis `DEL` on the cache key (cache.go:88–90). The order matters — deleting the DB row first and then invalidating the cache means a redirect that lands in the gap between those two steps would still see the (not-yet-

invalidated) cached value and serve a stale redirect for the link's remaining TTL window, which is the known trade-off of cache-aside invalidation versus a stricter write-through-with-lock scheme; this codebase accepts that window since a deleted link still briefly redirecting is low severity compared to adding write coordination to the hot path.

---

13. Optimization #3 — Collision-Free ID Generation

The problem: two approaches that look reasonable both have a real failure mode. Auto-incrementing in Postgres means every single shorten request needs a synchronous DB round trip just to get an ID, on top of the insert itself. And if two concurrent requests pick a custom alias independently (e.g. both check "is `/promo` taken?" then both insert), one of them wins and one gets a unique-constraint violation at best, or a silently overwritten link at worst, depending on how the insert is written.

Redis `INCR` + base62, collision-free by construction, not probabilistically (counter.go:7–20, base62.go): `NextID` calls Redis `INCR` on a single global counter key — `INCR` is atomic in Redis, so two concurrent callers are *guaranteed* distinct integers, never the same one. There's no hashing, no randomness, no birthday-paradox collision probability to reason about the way there would be with e.g. a random 6-character code — every counter value maps to exactly one base62 string and vice versa (`Encode` / `Decode` , base62.go:21–51), so uniqueness is a property of the counter being atomic, not a property of the encoding being "unlikely to repeat."

`SETNX`-based custom alias reservation and the exact race it prevents (cache.go:92–107, used from shorten.go:105–118): a custom alias isn't just checked against Postgres and then inserted — that would leave a window where two requests for the same alias both see "not taken" and both proceed to insert, with the database's unique constraint on `short_code` (schema.prisma:13) being the only thing stopping a duplicate, and even then only one request finds out it lost *after* doing the Postgres work. Instead, `ReserveAlias` does a Redis `SETNX` (`SET ... NX`) on `urlshortener:alias_reserved:<alias>` with a 30-second TTL (cache.go:96–102) — `SETNX` is atomic, so of two concurrent callers exactly one gets `true` back. The loser gets `store.ErrAliasTaken` immediately (shorten.go:114–116) — a fast `409` , no wasted Postgres work. The winner proceeds to the Postgres insert; if that insert fails for any other reason, `ReleaseAlias` (cache.go:105–107) frees the reservation immediately rather than waiting out the 30-second TTL (shorten.go:87–90).

★ interview tip — "Why 30 seconds for the reservation TTL, and why is it a safety net rather than the actual authority on whether an alias is taken?" Postgres's `UNIQUE` constraint on `short_code` is the real authority — the Redis reservation only needs to survive long enough to cover "SETNX, then one Postgres insert." Thirty seconds is comfortably longer than that round trip, so the TTL only matters if the process crashes between the two steps, in which case the alias becomes reservable again instead of being permanently stuck.

---

14. Optimization #4 — Decoupling Analytics from the Hot Path

The problem: a redirect is latency-sensitive in a way that writing an analytics row is not — the user is sitting there waiting for their browser to follow the `Location` header. If the redirect handler did a synchronous `INSERT INTO click_events` on every single hit, every redirect's latency would be bounded by Postgres write latency, and any Postgres slowdown (lock contention, a slow query from somewhere else, a failover) would directly slow down the one thing users are actually waiting on.

Redis Streams + consumer group, split across the producer (`PushClickEvent` , stream.go:43–61, called fire-and-forget from both the cache-hit and cache-miss branches of the redirect handler, redirect.go:35/61) and the consumer (`internal/worker/worker.go`):

- **Producer side:** `XADD urlshortener:click_events MAXLEN~100000 data=<json>` (stream.go:53–60). `MAXLEN ~ 100000` caps the stream so a long worker outage doesn't let it grow unbounded — it's described in the code as a safety net, not a normal operating ceiling (stream.go:49–51).
- **Consumer side:** the worker goroutine (worker.go:51–130, started in-process by `main.go:46–47 — there is no separate worker container) loops calling` ReadClickEvents `, which does` XGROUPCREATE … MKSTREAM `(idempotent, creates the group/stream on first use, stream.go:87) then` XREADGROUP GROUP analytics-workers … COUNT 100 BLOCK 5s `(stream.go:89–95). For each event it writes a row to Postgres (` CreateClickEvent `, worker.go:108), and only events that wrote successfully get collected into` ackIDs `and acknowledged with` XACK` (worker.go:119, 124–128, stream.go:119–124). A failed write is simply *not* acked, which leaves it in the consumer group's pending-entries list for redelivery rather than being dropped (worker.go:109–116) — this is explicitly how **at-least-once delivery** is achieved: nothing is acked until it's durably in Postgres.

Crash recovery: because unacked entries stay in the pending list tied to the consumer group (not to the specific worker process), if the worker process dies mid-batch, those events are still retrievable — a restarted worker (or a second one, since the group model supports multiple consumers) can claim them again. The worker's own `run()` loop additionally recovers from in-process panics (`defer recover()` , worker.go:52–59) and restarts itself rather than taking the whole process down with it — note the comment there: an unrecovered panic in *any* goroutine, including this one, would kill the entire Go process regardless of Fiber's own `recover()` middleware, since that only guards the HTTP-handling goroutines.

Why this makes analytics specifically "eventually consistent," and what that means in plain terms: a click is immediately durable in Redis (the stream) the instant it happens, but it isn't durable in Postgres — and therefore not visible to `/api/stats/:shortCode` or `/api/stats/top` — until the worker has drained it, acknowledged it, and committed the write, which happens on a 5-second poll cycle in batches of up to 100 (worker.go:32–34, 69–70). In plain terms: if you click a link and immediately check its stats, the click count might not include your click yet; check again a few seconds later and it will. The redirect itself, though, is never delayed by any of this — that's the entire point of the split.

15. Database Schema

`backend/prisma/schema.prisma` defines four tables, all accessed through `backend/internal/db/prisma_store.go` (the only implementation of the interfaces in `internal/store/store.go`).

`links` (model `Link`, schema.prisma:11–25)

Field	Type	Notes
`id`	BigInt, PK, autoincrement	
`short_code`	String, `@unique`	the redirect lookup key
`long_url`	String	
`user_id`	BigInt?, FK → `users.id`	nullable — anonymous shortens have no owner
`custom_alias`	Boolean, default false	
`expires_at`	DateTime?	checked in `GetLinkByShortCode` (prisma_store.go:106–109)
`password_hash`	String?	present in the schema/struct but no handler reads or sets it as a redirect gate — link-level password protection is modeled, not wired up
`created_at`	DateTime, default now()	

Indexes: `@@index([userId])` (schema.prisma:23) plus the implicit unique index backing `@unique` on `short_code`. The `userId` index exists because `GetUserLinks` (prisma_store.go:463–468) does `Link.UserID.Equals(...).OrderBy(Link.CreatedAt.Order(SortOrderDesc))` — without it, every "show me my links" call would be a full table scan. The `short_code` unique index is what makes `GetLinkByShortCode` (prisma_store.go:76–79), the redirect hot path's cache-miss fallback, an index lookup rather than a scan — this is the single most latency-critical query in the schema.

`users` (model `User`, schema.prisma:27–38)

`email` is `@unique` (schema.prisma:29) — this backs `GetOrCreateUserByEmail`'s `Upsert` (prisma_store.go:150–160), which relies on that constraint to make concurrent first-time signups race-safe rather than raising a duplicate-row bug. `password_hash` is stored but always set to `""` (prisma_store.go:156) — there's no password login in this system; the field is a schema leftover from before the magic-link model, noted directly in the code comment (prisma_store.go:154–155).

`verification_tokens` (model `VerificationToken`, schema.prisma:40–50)

`token_hash` is `@unique` (schema.prisma:42) — `GetVerificationTokenByHash` (prisma_store.go:199–202) looks up by it directly, and uniqueness also means two independently-generated tokens can never collide into the same lookup key. `@@index([userId])` (schema.prisma:48) isn't exercised by any query in `prisma_store.go` today (there's no "list a user's pending tokens" query) — it's defined for the FK relation but not yet used by a hot query; flag this as schema surface ahead of actual usage rather than a load-bearing index.

`api_keys` (model `ApiKey`, schema.prisma:52–62)

`key_hash` is `@unique` (schema.prisma:54), backing `GetAPIKeyByHash` (prisma_store.go:262–265) — this runs on **every single authenticated request** via `OptionalAPIKeyAuth` / `RequireAPIKeyAuth` (middleware/auth.go:37, 77), so it needs to be an index lookup, not a scan. `rate_limit_tier` defaults to `"standard"` (schema.prisma:57, mirrored in `CreateAPIKey`, prisma_store.go:242–244) and is what the rate limit middleware reads to pick a bucket (middleware/ratelimit.go:41). `@@index([userId])` (schema.prisma:60) exists for the FK relation but, like `verification_tokens`, has no direct query exercising it in `prisma_store.go` yet (there's no "list a user's API keys" endpoint).

`click_events` (model `ClickEvent`, schema.prisma:64–78)

Field	Notes
`link_id`	FK → `links.id`
`timestamp`	default now(), but in practice set explicitly by the worker to the event's actual click time (worker.go:100)
`referrer`, `country`, `device_type`, `ip_hash`	all nullable, best-effort

Three indexes here, each justified by a real query: `@@index([linkId])` (schema.prisma:74) backs `GetLinkStats`'s per-link fetch (`ClickEvent.LinkID.Equals(...)`, prisma_store.go:334–336) and `DeleteLink`'s cascade-style cleanup (`ClickEvent.LinkID.Equals(...).Delete()`, prisma_store.go:127–131, needed because Postgres would otherwise reject the link delete on the FK constraint). `@@index([timestamp])` (schema.prisma:75) isn't directly exercised by a `WHERE timestamp` predicate anywhere in `prisma_store.go` today — the daily-grouping logic in `GetLinkStats` (prisma_store.go:376–388) buckets by day in Go after fetching all events for a link, not via a timestamp-range query. The composite `@@index([linkId, timestamp])` (schema.prisma:76) is what actually backs `GetRecentClickEvents` (`ClickEvent.LinkID.Equals(...).OrderBy(ClickEvent.Timestamp.Order(SortOrderDesc)).Take(limit)`, prisma_store.go:510–514) — filter by `link_id` and sort by `timestamp` without a separate in-memory sort, which the code's own comment calls out directly ("backed by the ClickEvent(linkId, timestamp) composite index … rather than fetching everything and sorting in Go," prisma_store.go:507–509).

Worth flagging as a real finding, not just description: `GetLinkStats` (prisma_store.go:322–397) and `GetTopLinks` (prisma_store.go:400–459) both load **every** click event for a link (or for all links, in `GetTopLinks`'s case — `s.client.ClickEvent.FindMany().Exec(ctx)` with no filter at all, prisma_store.go:416) into memory and aggregate in Go, rather than using `COUNT` / `GROUP BY` at the database level. The `timestamp` index exists but isn't leveraged by either of these — they're full per-link (or full-table) reads followed by Go-side counting and insertion sort. This works fine at current data volumes but is the clearest N+1-adjacent / scale-limiting pattern in the analytics read path, and the index design on `click_events` is ahead of what the queries actually use.

Part 3 — The Frontend and the User Journey

We've spent two parts underground, in Go handlers and Redis streams. Now we surface. This part is about the thing a human actually looks at: a React SPA that talks to all of that machinery through exactly one door. We'll walk the component tree, find that one door, meet the two bits of global state that hold the app together, and then tell the whole story end to end — a stranger showing up with no account, and leaving with a working short link and a dashboard full of stats.

---

16. The component tree

Everything starts at `frontend/src/main.jsx`. It's four lines of nesting that matter:

```
<BrowserRouter>
  <ToastProvider>
    <AuthProvider>
      <App />
    </AuthProvider>
  </ToastProvider>
</BrowserRouter>
```

(`frontend/src/main.jsx:11–17`). Order here isn't decorative — `ToastProvider` wraps `AuthProvider`, which means anything auth does (including, as we'll see, nothing on login failure) *could* reach for a toast, and does. `App` sits innermost, so it's the one place that actually owns the page layout.

`App.jsx` itself is almost pure routing (`frontend/src/App.jsx:13–45`):

```
<div> (flex column, min-h-screen)
 ├─ <Navbar />
 ├─ <main>
```

```
  │    <Routes>
  │      "/"              → <LandingPage />
  │      "/login"         → <LoginPage />
  │      "/top"           → <TopLinksPage />
  │      "/dashboard"     → <ProtectedRoute><DashboardPage /></ProtectedRoute>
  │      "/stats/:shortCode" → <ProtectedRoute><StatsPage /></ProtectedRoute>
  │      "/admin"         → <ProtectedRoute><AdminPage /></ProtectedRoute>
  │    </Routes>
  ├─ <Footer />
  └─ <Toast />
```

Three public routes, three gated ones. `ProtectedRoute` (`frontend/src/components/ProtectedRoute.jsx:4-12`) is deliberately tiny — it reads `isAuthenticated` off `useAuth()` and either renders `children` or `<Navigate to="/login" replace />`. No loading state, no role check — it's a boolean gate, nothing more. Worth flagging early: this gate is just hiding UI. `/admin` is in that list, but nothing here or in the Go backend ties "has an API key" to "is allowed to see system metrics" — any authenticated user who manually fetches `/api/metrics` or visits `/admin` gets the same system-wide dashboard. The route is "protected" from anonymous visitors, not from ordinary users.

Here's the full tree, with what each leaf actually renders and calls:

```
App
├─ Navbar                          (frontend/src/components/Navbar.jsx)
│    state: mobileMenuOpen
│    reads: useAuth() → isAuthenticated, logout
│    no API calls — theme toggle just flips a DOM class + localStorage.theme
│
├─ Routes
│    ├─ LandingPage                (pages/LandingPage.jsx)
│    │    state: stats {totalLinks, totalClicks}, loadingStats
│    │    calls: GET /api/stats/top?limit=100  (on mount, derives totals client-
side)
│    │    renders: ShortenForm, StatCard ×2, FeatureCard ×3 (local, non-exported)
│    │
│    ├─ LoginPage                  (pages/LoginPage.jsx)
│    │    state: email, loading, verificationSent, existingKey, emailFocused
│    │    reads: useAuth() → requestVerification, importKey; useToast()
│    │    calls: requestVerification() → POST /api/keys {email}  (indirect, via
AuthContext)
│    │    no direct API call for the "import key" form — it's pure client-side
│    │    regex validation (isValidApiKey) then setApiKey
│    │
│    ├─ TopLinksPage                (pages/TopLinksPage.jsx)
│    │    state: links, loading
│    │    calls: GET /api/stats/top?limit=10   (public, unauthenticated)
│    │    renders: Spinner, plain <Link> to /stats/:shortCode per row
│    │
│    ├─ ProtectedRoute → DashboardPage  (pages/DashboardPage.jsx)
```

```
│    │    state: links, loading, searchTerm, showForm, searchFocused
│    │    reads: useAuth() → apiKey; useToast()
│    │    calls: GET /api/links  (on mount, re-run whenever apiKey changes)
│    │           DELETE /api/links/:shortCode  (on delete click)
│    │    renders: ShortenForm (conditionally, toggled by showForm),
│    │             LinkTable (filtered client-side by searchTerm), Spinner
│    │
│    ├─ ProtectedRoute → StatsPage      (pages/StatsPage.jsx)
│    │    params: :shortCode (from the URL)
│    │    state: stats, clicks, loading
│    │    calls (parallel, Promise.all): GET /api/stats/:shortCode
│    │                                   GET /api/links/:shortCode/clicks?limit=20
│    │    renders: StatCard ×2, DailyChart, plus an inline referrers list
│    │             and a recent-clicks table built directly in this file
│    │
│    └─ ProtectedRoute → AdminPage      (pages/AdminPage.jsx)
│         state: metrics, loading
│         calls: GET /api/metrics
│         renders: StatCard ×4, a Recharts PieChart (status codes),
│                  a hand-built cache hit-rate progress bar,
│                  two hit/miss stat tiles
│
├─ Footer                          (components/Footer.jsx) — static, no state, no
calls
└─ Toast                           (components/Toast.jsx)
     reads: useToast() → toasts, removeToast
     renders one animated card per toast in a fixed bottom-right stack
```

Shared leaf components, used by more than one page: - `ShortenForm` (`components/ShortenForm.jsx`) — used by `LandingPage` and `DashboardPage` . Owns its own `url` / `customAlias` / `expiresAt` / `result` state; calls `POST /api/shorten` directly (not through a context). - `LinkTable` (`components/LinkTable.jsx`) — pure presentational, takes `links` and `onDelete` as props, no API calls of its own. - `StatCard` (`components/StatCard.jsx`) — pure presentational, a tilt-on-hover number tile; no API calls. - `Spinner` (`components/Spinner.jsx`) — pure presentational. - `DailyChart` (`components/DailyChart.jsx`) — pure presentational, wraps Recharts' `BarChart` around a `data` prop.

★ interview tip > *"Why does `ProtectedRoute` only check `isAuthenticated` , and why can any > logged-in user reach `/admin` ?"* > Because the frontend's auth model has exactly one tier below "has a valid > key" — there's no role or scope encoded in the API key itself (see Part 2's > auth chapter), so there's nothing finer-grained for the route guard to > check. The fix isn't a frontend one: the backend would need a `role` or > `admin` flag on the user record, returned from some "who am I" endpoint, > before `ProtectedRoute` could meaningfully distinguish an admin from any > other authenticated user.

---

17. How the UI and backend actually talk

Every single HTTP request this frontend makes — all ten-ish call sites you saw in the tree above — goes through one file: `frontend/src/api/client.js`. It's 48 lines, so let's take it in full.

```
const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_URL || '',
  headers: { 'Content-Type': 'application/json' },
});
```

(`frontend/src/api/client.js:3-8`). `VITE_API_URL` is the only environment knob. In dev it's typically unset, which means `baseURL` is `''` and every call is a relative path — Vite's dev server proxies `/api`, `/health`, and `/metrics` to `:8080` (per the codebase guide), so `apiClient.get('/api/links')` just works without CORS ever entering the picture. In production the SPA is served *by* the same Go binary it's calling, so relative paths work there too — `VITE_API_URL` only matters if you ever split the frontend onto its own origin.

The request interceptor is the entire authentication story on the client side:

```
apiClient.interceptors.request.use((config) => {
  const apiKey = localStorage.getItem('urlshortener_api_key');
  if (apiKey) {
    config.headers.Authorization = `Bearer ${apiKey}`;
  }
  return config;
});
```

(`frontend/src/api/client.js:10-19`). Every outgoing request — authenticated or not — passes through here and gets the key attached *if one exists in localStorage*. There's no in-memory source of truth checked first; the interceptor reads directly from storage on every call. That's notable because it means `AuthContext`'s `apiKey` state and `localStorage` have to stay in sync by convention (they do — see Chapter 18), not because the interceptor enforces it.

The response interceptor is where the app's error handling actually lives:

```
apiClient.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response) {
      const { status, data } = error.response;
      if (status === 401) {
        localStorage.removeItem('urlshortener_api_key');
        if (window.location.pathname !== '/login') {
          window.location.href = '/login';
        }
      }
```

```
      let errorMessage = data?.error || data?.message || 'An unexpected error
        occurred';
      if (status === 429) errorMessage = 'Too many requests. Please wait a moment
        and try again.';
      else if (status === 500) errorMessage = 'Server error. Please try again
        later.';
      return Promise.reject({ status, message: errorMessage, data });
    }
    return Promise.reject({ status: 0, message: 'Network error. Please check your
        connection.' });
  }
);
```

(`frontend/src/api/client.js:21–46`). Two things worth calling out:

1. **The 401 handler is a hard `window.location.href` redirect**, not a React Router navigation. That means a full page reload on every session-expiry — simple, reliable, and it blows away any in-flight app state, which is arguably fine for an app this size.
2. **Every call site gets a normalized error shape**: `{status, message, data}`, never a raw axios error. That's why you see the same pattern everywhere — `catch (error) { showToast(error.message || '...', 'error'); }` — in `LoginPage`, `DashboardPage`, `StatsPage`, `TopLinksPage`, `AdminPage`. The interceptor did the translation once so every page could skip it.

So: `client.js` is simultaneously the auth-header injector, the session-expiry watchdog, and the error-message translator. Nothing downstream — not a single page or component — talks to `fetch` or raw `axios` directly. If you wanted to add request tracing, a loading-progress bar, or retry-on-5xx, this is the one file you'd touch.

★ interview tip > *"What happens if the API key in localStorage is stale (revoked > server-side) and the user is sitting on the dashboard?"* > The next request that hits `/api/links` or similar gets a 401 back. The > response interceptor clears `urlshortener_api_key` from storage and does a > full `window.location.href = '/login'`. `AuthContext`'s `apiKey` state > doesn't get cleared through its own `logout()` here — it's bypassed entirely because the redirect reloads the page, remounting `AuthProvider` from scratch, which re-reads `localStorage` (now empty) in its `useState` initializer (`context/AuthContext.jsx:7`). So the state ends up correct, just via a full remount rather than a state update.

---

18. AuthContext and ToastContext

These are the only two pieces of state shared across more than one branch of the tree — everything else (`links` , `stats` , `metrics` , form inputs) is local to the page that fetched it.

AuthContext (`frontend/src/context/AuthContext.jsx`) holds exactly two pieces of state:

```
const [apiKey, setApiKey] = useState(localStorage.getItem('urlshortener_api_key')
        || null);
const [isAuthenticated, setIsAuthenticated] = useState(!!apiKey);
```

(lines 7-8). `isAuthenticated` is derivable from `apiKey` and is kept in sync by an effect rather than computed inline:

```
useEffect(() => {
  if (apiKey) {
    localStorage.setItem('urlshortener_api_key', apiKey);
    setIsAuthenticated(true);
  } else {
    localStorage.removeItem('urlshortener_api_key');
    setIsAuthenticated(false);
  }
}, [apiKey]);
```

(lines 10-18). This is the *other* half of the localStorage sync mentioned in Chapter 17 — the effect writes to storage whenever `apiKey` changes in React state, while the axios interceptor reads from storage independently. Two different code paths agreeing on the same key name (`urlshortener_api_key`) is doing the work that a single shared accessor would normally do.

Three functions are exposed through the context value (`context/AuthContext.jsx:39`): - `requestVerification(email)` — `POST /api/keys {email}`, returns the backend's message. This is the "send me a magic link" call. Note the comment right above it in the source (lines 20-24): the backend used to return a key directly from this endpoint; now it only sends an email, and the key only becomes available once the link is clicked. - `importKey(key)` — just `setApiKey(key)`. This is how a freshly-obtained key (pasted in after clicking the magic link, or typed in directly) enters the app's state. - `logout()` — `setApiKey(null)`, which cascades through the effect above to clear storage and flip `isAuthenticated`.

Consumers: `Navbar` (`isAuthenticated`, `logout`), `ProtectedRoute` (`isAuthenticated`), `LoginPage` (`requestVerification`, `importKey`), `DashboardPage`, `StatsPage`, `AdminPage` (all just read `apiKey`, to key their `useEffect` fetches).

ToastContext (`frontend/src/context/ToastContext.jsx`) holds one array:

```
const [toasts, setToasts] = useState([]);
```

with `showToast(message, type = 'info', duration = 4000)` appending a `{id, message, type}` object and scheduling its own removal via `setTimeout` (lines 8-15), and `removeToast(id)` for the manual dismiss (the `X` button in `Toast.jsx`). Nothing else holds a reference to a toast after it's created — no queue limit, no dedup. Every page that can fail a network call calls `showToast` from its `catch` block; `Toast.jsx` is the only consumer that reads the `toasts` array itself, rendering one animated card per entry.

What actually triggers a change in each context: - **AuthContext** changes on: a successful `importKey` call (either from pasting a magic-link key, or — see Chapter 19 — any flow that produces a key), and on `logout()` (Navbar's logout button) or the implicit logout caused by a 401 reload (Chapter 17). - **ToastContext** changes on: any `catch` block across five pages/components, three explicit success toasts (`ShortenForm` on shorten, `LoginPage` on verification sent, `LinkTable` / `ShortenForm` on copy-to-clipboard), and the self-expiring `setTimeout` inside `showToast` itself.

★ **interview tip** > *"Why does `AuthContext` need both `apiKey` and `isAuthenticated` if one is* > *derivable from the other?"* > It doesn't strictly need both — `isAuthenticated` is just `!!apiKey` kept > as a mirrored boolean via `useEffect` rather than computed at each call > site. It's a readability/ consistency choice (every consumer that only > cares about "logged in or not" doesn't have to know the field is a key), > not a correctness requirement. It does cost one extra render per `apiKey` > change, since the effect fires after the state update.

---

19. The full user journey

Picture someone — call her Priya — who has never seen this app before. She has a link she wants to shrink and hand to a client. Here's what happens, file by file.

She opens the root URL. React mounts `main.jsx`'s provider stack (`frontend/src/main.jsx:11–17`): `ToastProvider` first, `AuthProvider` second — and `AuthProvider`'s very first act, before anything renders, is to check `localStorage.getItem('urlshortener_api_key')` (`context/AuthContext.jsx:7`). Priya's never been here, so that's `null`, and `isAuthenticated` starts `false`. `App.jsx` routes `"/"` to `LandingPage` (`App.jsx:19`). On mount, `LandingPage` fires `GET /api/stats/top?limit=100` through `apiClient` (`pages/LandingPage.jsx:24–25`) — note this call carries no `Authorization` header, because the request interceptor only *adds* one if `localStorage` has a key (`api/client.js:12–14`), and public aggregate stats don't need one. The response feeds two `StatCard`s showing "Links Shortened" and "Total Clicks Tracked" (`pages/LandingPage.jsx:149–167`) — social proof, not anything Priya needs yet.

What Priya actually wants is a login. She clicks "Login" in the `Navbar` (rendered because `isAuthenticated` is false, so the nav shows the "Login" link rather than a logout button — `components/Navbar.jsx:128–141`), landing on `LoginPage`. She types her email and submits the first form. That calls `handleRequestVerification` (`pages/LoginPage.jsx:21–35`), which awaits `requestVerification(email)` from `AuthContext` — which is just `apiClient.post('/api/keys', { email })` (`context/AuthContext.jsx:26`). The backend's response here is deliberately just a message, not a key (per the comment at `AuthContext.jsx:20–24`) — it has gone off to send her a magic link instead. `LoginPage` flips `verificationSent` to `true` (`pages/LoginPage.jsx:28`), which swaps the form out for a "Check your email" card (`pages/LoginPage.jsx:64–108`), and a success toast fires (`'Verification email sent!'`, line 29).

In dev, that email landed in MailHog, and the UI is honest about it — the confirmation card literally links to `localhost:8025` (`pages/LoginPage.jsx:88–93`) so Priya doesn't have to go hunting for it.

She opens MailHog, finds the magic link, and clicks it. That GET request is outside the SPA entirely — it hits the backend directly (`GET /api/keys/verify?...` per the `AuthContext` comment), which validates the 15-minute token, generates an API key of the form `usk_<64 hex>`, and shows it to her — once, as the codebase guide puts it; only its SHA-256 hash persists server-side. That key never lives in the React app yet; Priya has to carry it back herself.

She goes back to the login tab and pastes the key into the second form — "Already have a key?" (`pages/LoginPage.jsx:183–205`). Submitting calls `handleImportKey` (lines 37-46), which runs the key through `isValidApiKey` — a plain regex, `/^usk_[a-f0-9]{64}$/` (`utils/format.js:40–42`) — purely client-side, no network round trip to check it's real. If it matches the shape, `importKey(existingKey)` is called, which is `AuthContext`'s `setApiKey` (`context/AuthContext.jsx:30–32`). That state change triggers the `useEffect` at `AuthContext.jsx:10–18`, which writes the key into `localStorage` under `urlshortener_api_key` and flips `isAuthenticated` to `true`. `LoginPage` then calls `navigate('/dashboard')` (`pages/LoginPage.jsx:45`). Notice: the app never actually confirms the key is *valid* against the server at this moment — the first real test of the key is whatever `DashboardPage` fetches next.

`DashboardPage` mounts, sees a non-null `apiKey` from `useAuth()`, and its effect fires `fetchLinks()` → `GET /api/links` (`pages/DashboardPage.jsx:30–46`) — this request does carry `Authorization: Bearer usk_...` now, attached invisibly by the request interceptor. If the key were bogus, this is where a 401 would come back and the response interceptor would bounce her straight to `/login` (`api/client.js:27–31`). It isn't bogus, so she gets an (empty, for now) list back, and `LinkTable` renders its empty state: "No links found. Shorten your first URL!" (`components/LinkTable.jsx:19–29`).

She clicks "Shorten New URL" (`pages/DashboardPage.jsx:77–87`), which toggles `showForm` and animates in a `ShortenForm` (`pages/DashboardPage.jsx:90–105`). She pastes her long URL, optionally opens "Advanced Options" to set a custom alias or expiry (`components/ShortenForm.jsx:107–154`), and submits. `handleSubmit` (`components/ShortenForm.jsx:19–44`) builds a body — `{url, custom_alias?, expires_at?}` — and calls `apiClient.post('/api/shorten', body)`. The response becomes `result`, rendered as a green "Your shortened URL" card with a copy button (`components/ShortenForm.jsx:157–189`); `onSuccess` is also invoked, which on the dashboard means `fetchLinks()` runs again and the form collapses (`pages/DashboardPage.jsx:99–102`) — so the new link shows up in the table underneath, not just in the ephemeral result card.

Now she wants to see how it's doing. Back on the dashboard, her new row in `LinkTable` has a chart icon linking to `/stats/:shortCode` (`components/LinkTable.jsx:91–99`). Following it mounts `StatsPage`, whose effect fires two requests in parallel via `Promise.all` (`pages/StatsPage.jsx:35–38`): `GET /api/stats/:shortCode` for aggregate numbers, and `GET /api/links/:shortCode/clicks?limit=20` for the raw recent hits. The aggregate response populates two `StatCard`s (total clicks, unique visitors — `pages/StatsPage.jsx:102–103`), a `DailyChart` bar chart fed by `stats.daily_clicks` (`pages/StatsPage.jsx:106–109`, chart itself in `components/DailyChart.jsx`), and a referrers list built inline right there in `StatsPage` (lines 117-138). The per-click response fills the "Recent Clicks" table, also inline in the same file (lines 141-183), each row

showing a formatted timestamp (`formatDate` , `utils/format.js:1–11`), country, and a device icon swapped by type (`pages/StatsPage.jsx:50–56`). Right now, with zero clicks, both are empty — "No referrer data available," "No recent clicks."

So she copies the short link (the copy button in `ShortenForm` 's result card, or from `LinkTable` — both just call `navigator.clipboard.writeText` and fire a success toast, `components/ShortenForm.jsx:46–51` / `components/LinkTable.jsx:12–17`) and sends it to her client. The client — a third party who never touches this React app at all — clicks the link in their browser. That request goes straight to the Go backend's redirect handler; nothing in the SPA is involved (see Chapter 20, and Part 2's Chapter 9 for the server-side mechanics of that hit).

Some time later, Priya comes back, revisits `/stats/that-code` , and `StatsPage` 's effect runs again (it re-fires whenever `shortCode` or `apiKey` change — `pages/StatsPage.jsx:48` — and a fresh mount counts as that). The same two requests go out, but now `total_clicks` is 1, `unique_ips` is 1, the `DailyChart` has a bar, and the "Recent Clicks" table has her client's hit — country and device type included, assuming the backend's click pipeline (Part 2) finished processing it by the time she checked. Nothing pushed that update to her proactively — there's no polling or websocket here, just a fresh fetch on each visit.

★ interview tip > *"Where exactly does the frontend ever see or persist the raw API key, > beyond pasting it into the import form?"* > Exactly once per login: as the string the user manually copies from the > magic-link landing page and pastes into `LoginPage` 's second form > (`pages/LoginPage.jsx:188–195`). From there it's `importKey` → `setApiKey` → the `AuthContext` effect writes it to `localStorage.urlshortener_api_key` (`context/AuthContext.jsx:12`), and the axios request interceptor reads it back out of storage on every subsequent call (`api/client.js:12`). It is never kept purely in memory-only state in a way that would survive a key *not* being in localStorage — refreshing the page re-derives `isAuthenticated` from storage, not from any server session.

---

20. What happens when the third party clicks the link — from the browser's side

Zoom out from the React app entirely for a moment, because the person who clicks Priya's short link never loads this SPA at all. From their browser's point of view, the sequence is almost comically plain:

1. **DNS/TLS** — the browser resolves whatever domain the short link is hosted on and negotiates TLS if it's HTTPS. This is entirely outside this application's control; the Go binary doesn't participate in either step beyond possibly sitting behind a reverse proxy that handles TLS termination (see the deployment docs for how nginx fits into that, where applicable).
2. **The GET request** — the browser sends a plain `GET /{shortCode}` to whatever host answers. No cookies, no special headers, nothing the SPA set up — this is just a normal top-level navigation, indistinguishable from typing a URL directly.

3. **The 302 response** — the server replies with an HTTP redirect (`Location: <long url>`) rather than any HTML page. The browser never renders anything from this app's frontend bundle for this request — there's no `index.html`, no JS, no React involved at any point in a short link visit.
4. **The browser's own redirect behavior** — on receiving the 3xx, the browser transparently issues a second request to the `Location` target and navigates there. The user sees only the destination site load; the short-link hop is invisible unless they're watching the network tab or the URL bar flickers momentarily.

What the user never sees behind that redirect is everything Part 2's Chapter 9 already walked through in detail: the Redis cache lookup keyed on the short code (cache hit returning the `{url, id}` JSON immediately, or a Postgres fallback on miss with a cache backfill), and the fire-and-forget push of a click event onto the Redis Stream for the analytics worker to pick up later. None of that touches the response the browser gets — the 302 goes out either way, and the click-event write happens asynchronously, off the hot path. That asynchronous write is exactly what turns into the "Recent Clicks" row and the incremented `total_clicks` that Priya saw when she checked back in Chapter 19 — the browser's redirect and the dashboard's updated numbers are two completely decoupled consequences of the same single GET request, joined only by the click-event stream in between.

★ interview tip > *"If the click-event write to the Redis Stream fails, does the visitor's > redirect fail too?"* > No — by construction they can't be coupled. The redirect response is > already determined by the cache/DB lookup before the click event is > pushed, and the push itself is fire-and-forget. A failed or delayed event > write only affects whether that click ever shows up in `StatsPage`'s > numbers; it has zero effect on whether the third party's browser gets > redirected. That's the whole point of decoupling analytics from the redirect hot path (Part 2, Chapter 9).

Part 4 — Monitoring, CI/CD, Deployment, and Load Testing

The first three parts (wherever they live) presumably covered the shortener itself: the redirect hot path, the auth model, the data layer. This part covers everything that turns "a Go binary that works on my machine" into "a thing with a public URL, a dashboard, and an automated pipeline that redeploys it." That's nginx, Prometheus/Grafana, GitHub Actions, and a real (if modest) load test against a real (if temporary) deployment.

One thing up front, because it shapes how to read everything below: this repo's own `DEPLOY.md` and several of its `docs/system-design/adr/*.md` files describe an *earlier* deployment story — a Windows+WSL2 self-hosted runner, an Oracle/Render/Cloud-Run provider menu, a CD pipeline that was "accepted, implemented, then reverted." None of that is what's running today. The actual commit history (`git log --oneline --all`, checked while writing this) shows the deploy workflow came back on October 15, 2026 (`8d97404`), and the real deploy target turned out to be an Apple Silicon (arm64) machine, not Windows — confirmed by commit `6e67a8f`'s own message, quoted in

full in Chapter 25. Per this project's own ground rule (§36, `CLAUDE.md`): code and git history are the source of truth, not stale docs. Where the two disagree below, that's flagged explicitly rather than quietly picking one.

---

21. nginx — why a reverse proxy sits in front of the API at all

The problem it solves

Without nginx, `docker-compose.prod.yml` would have no choice but to publish the Go API container's port 8080 straight to the internet. That's not hypothetical — look at what the prod overlay actually does to every *other* service:

```
# docker-compose.prod.yml:58-64
postgres:
  ports: !reset []
redis:
  ports: !reset []
```

Postgres and Redis lose their host port mappings entirely in production. They're reachable only over the internal Docker network, by service name. The API can't disappear the same way — something has to be public, or there's no product. nginx is that something: one container whose only job is to be the public entrypoint, so the app server itself never has to be.

That buys three concrete things without writing a line of Go:

1. **A single, auditable point of public exposure.** One container, one config file, one thing to reason about when asking "what can the internet reach?"
2. **Headers and policy at the edge, for free.** Security headers, gzip, routing — done once in nginx config instead of as middleware in every backend language/framework that might ever sit behind it.
3. **A seam for TLS later**, without having to touch the Go binary. TLS termination is a proxy-layer concern; putting nginx in front now means adding HTTPS later is a config change at the edge, not an application change.

The alternative — exposing Fiber directly — isn't wrong in the sense of "won't run." It's wrong in the sense that TLS, security headers, and origin-handling logic would all have to be either hand-rolled in Go middleware or simply absent. A reverse proxy is the industry's standard answer to "give me these things without writing them."

Walking `nginx/nginx.conf` line by line

```
worker_processes auto;          # one worker per CPU core — nginx scales
                                # itself to the host, no tuning needed here
```

```
events {
    worker_connections 1024;    # max simultaneous connections per worker
}

http {
    include       /etc/nginx/mime.types;   # correct Content-Type by file ext
    default_type  application/octet-stream;

    sendfile        on;         # kernel-level file send, skips a userspace copy
    keepalive_timeout  65;      # how long to hold idle keep-alive connections

    gzip on;
    gzip_types text/plain text/css application/json application/javascript
               text/xml application/xml application/xml+rss text/javascript;
    gzip_min_length 256;        # don't bother compressing tiny responses

    upstream api_backend {
        server api:8080;        # Docker service name, not localhost — resolved
                                 # via Docker's embedded DNS on the compose network
    }

    server {
        listen 80;
        server_name _;          # catch-all: respond regardless of Host header
```

That `upstream api_backend { server api:8080; }` block is worth pausing on, because it's the clearest instance in the whole stack of the `CLAUDE.md` Docker-networking rule (§4): "services must communicate using Docker networking/service names rather than incorrectly using `localhost`." `api` here isn't a hostname that happens to work — it's the literal Docker Compose service name, resolved by Docker's internal DNS. On the host machine, `api` means nothing; inside the compose network, it's the only address nginx needs for the whole backend.

Security headers, applied to every response:

```
add_header X-Frame-Options "DENY" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
add_header Content-Security-Policy "default-src 'self'; img-src 'self' data:;
        script-src 'self'; style-src 'self' 'unsafe-inline'; connect-src 'self'"
        always;
```

- `X-Frame-Options: DENY` — this site can never be framed by another site. Kills classic clickjacking.
- `X-Content-Type-Options: nosniff` — stops browsers from guessing a different MIME type than the one the server declared, which closes off a class of content-sniffing XSS.
- `Referrer-Policy: strict-origin-when-cross-origin` — don't leak full URLs (which, for a URL shortener, could include meaningful path segments) to third-party `Referer` headers on cross-origin navigation.

- The CSP is a real allowlist, not a report-only stub: scripts, styles, images, and XHR/fetch (`connect-src`) are all restricted to `'self'` (images additionally allow `data:` URIs). `style-src` needs `'unsafe-inline'` — consistent with a Vite/Tailwind-built SPA that injects inline styles at runtime.

The proxy block itself:

```
location / {
    proxy_pass http://api_backend;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

This is the part that does the actual forwarding, and the four `proxy_set_header` lines are what make the forwarding *honest*. Without them, every request the Go app sees would appear to originate from nginx itself (`127.0.0.1` or the Docker bridge IP), not the real client:

- `Host $host` — preserves the original Host header the client sent.
- `X-Real-IP $remote_addr` — the immediate connecting IP (nginx's own view).
- `X-Forwarded-For $proxy_add_x_forwarded_for` — appends to any existing forwarding chain rather than overwriting it, so a chain of proxies stays traceable.
- `X-Forwarded-Proto $scheme` — tells the backend whether the original request was http or https, which matters once TLS terminates upstream of nginx (see below) and the Go app needs to know the client actually used HTTPS even though the hop it sees is plain HTTP.

This is a concrete instance of "things a proxy does for free": reproducing this logic in Go middleware is possible, but it's the kind of boilerplate every single backend behind every single reverse proxy in the world re-implements identically — which is exactly the kind of thing a proxy layer exists to centralize.

Explicit on TLS: nginx here is HTTP-only

`nginx.conf` has exactly one `listen` directive — `listen 80;` — and no `ssl_certificate` anywhere in the file. There is no TLS termination inside this container, deliberately. `DEPLOY.md`'s "Notes" section spells out the actual plan:

> use a **Cloudflare Tunnel** (`cloudflared tunnel --url http://localhost:80` …): it's outbound-only from the runner machine … and terminates TLS at Cloudflare's edge — so nginx itself can stay HTTP-only as configured.

The same applies to the other environment this project runs in, GitHub Codespaces, whose own edge handles HTTPS for the forwarded port before traffic ever reaches this container. Either way, nginx's

job stops at being the single internal entrypoint; TLS is somebody else's edge, by design — ADR-0005 confirms this explicitly: "TLS termination is not yet configured — deployment-target-dependent."

> **★ interview tip Q: If nginx has no TLS config, isn't that site serving plaintext HTTP?** A: From nginx's own perspective, yes — it only speaks HTTP on port 80. But that's not the end of the path a real client takes. In this project's actual operating mode, the public-facing hop (Cloudflare's tunnel edge, or GitHub Codespaces' own edge) terminates HTTPS *before* traffic reaches this machine at all, and reaches nginx over the tunnel's own encrypted channel. So the browser-to-edge leg is HTTPS; the edge-to-nginx leg, running over a private tunnel on the operator's own machine rather than the open internet, is HTTP. That's a legitimate, common pattern (it's exactly how most CDN/tunnel setups work) — but it's worth being able to say precisely *where* TLS terminates rather than asserting "it's HTTPS" as a blanket claim about the whole path.

> **★ interview tip Q: Why have nginx at all if Cloudflare's tunnel is already acting as a proxy?** A: The tunnel solves "how do I get traffic from the internet to my machine without a static IP or opened router ports" — it's a network-layer concern. nginx solves a different layer: once traffic *is* on the machine, what single internal entrypoint enforces headers, gzip, and routing before anything touches the app? Swap the tunnel for a VM with a real public IP tomorrow and nginx's job is unchanged; the tunnel is incidental to how traffic currently arrives, nginx is structural to how it's handled once it does.

---

22. Metrics, and what they actually tell you

There are genuinely two separate metrics surfaces in this codebase, and they serve two different consumers. Worth being precise about both, because it's easy to conflate them.

`/metrics` — Prometheus exposition format

Defined in `backend/internal/metrics/prometheus.go` (71 lines), registered via `promauto` (safe to call more than once — idempotent registration). These are the *real* metric names, copied verbatim, not paraphrased:

```
// backend/internal/metrics/prometheus.go
HTTPRequestsTotal = promauto.NewCounterVec(
    prometheus.CounterOpts{Name: "urlshortener_http_requests_total", ...},
    []string{"method", "path", "status"},
)

HTTPRequestDuration = promauto.NewHistogramVec(
    prometheus.HistogramOpts{
        Name: "urlshortener_http_request_duration_seconds",
        Buckets: prometheus.DefBuckets,
    },
```

```
    []string{"method", "path"},
)

RedirectsTotal = promauto.NewCounterVec(
    prometheus.CounterOpts{Name: "urlshortener_redirects_total", ...},
    []string{"result"},
)

RateLimitTotal = promauto.NewCounterVec(
    prometheus.CounterOpts{Name: "urlshortener_rate_limit_total", ...},
    []string{"result"},
)

AnalyticsEventsProcessed = promauto.NewCounter(
    prometheus.CounterOpts{Name:
        "urlshortener_analytics_events_processed_total", ...},
)
AnalyticsEventsFailed = promauto.NewCounter(
    prometheus.CounterOpts{Name:
        "urlshortener_analytics_events_failed_total", ...},
)
AnalyticsProcessingDuration = promauto.NewHistogram(
    prometheus.HistogramOpts{
        Name: "urlshortener_analytics_processing_duration_seconds",
        Buckets: prometheus.DefBuckets,
    },
)
```

Six metrics total, all namespaced `urlshortener_*`. Two are label-free (`AnalyticsEventsProcessed` / `Failed`), the rest carry labels sized to stay low-cardinality — `method` / `path` / `status` on the HTTP ones, `result` on redirects and rate-limit decisions. That "`path`, not raw URL" choice is deliberate and documented right in the middleware that populates it:

```
// backend/internal/middleware/prometheus.go:19-23
// Resolve the registered route path (not the raw URL)
routePath := c.Route().Path
if routePath == "" {
    routePath = c.Path()
}
```

`c.Route().Path` is Fiber's *registered* route pattern — `/:shortCode`, not `/gh-mirror` or `/abc123`. If this instead recorded the raw path, every distinct short code ever visited would mint a new Prometheus time series forever, a textbook cardinality explosion that would eventually make the `/metrics` endpoint itself slow and Prometheus's storage grow unboundedly. Using the route pattern caps the `path` label to the fixed set of routes the app actually registers, regardless of how many short links exist.

The middleware wires both metrics on every request:

```go
// backend/internal/middleware/prometheus.go:11-31
func PrometheusMetricsMiddleware() fiber.Handler {
    return func(c *fiber.Ctx) error {
        start := time.Now()
        err := c.Next()
        routePath := c.Route().Path
        if routePath == "" { routePath = c.Path() }
        method := c.Method()
        status := strconv.Itoa(c.Response().StatusCode())
        metrics.HTTPRequestsTotal.WithLabelValues(method, routePath, status).Inc()
        metrics.HTTPRequestDuration.WithLabelValues(method,
         routePath).Observe(time.Since(start).Seconds())
        return err
    }
}
```

This is Prometheus's text exposition format — the standard a Prometheus server scrapes directly, line-oriented `metric_name{labels} value`, built by the `promhttp` handler the Prometheus client library provides. It's machine-first: meant to be pulled by Prometheus on an interval, not read by a human in a browser tab (though you can, and it's a reasonable way to sanity-check what's actually being recorded).

`/api/metrics` — JSON, for the admin dashboard

A completely separate, hand-rolled metrics system lives in `backend/internal/metrics/metrics.go` (92 lines): atomic counters (`atomic.Int64`), no external dependency, a `Snapshot()` method that returns a `map[string]interface{}` — uptime, total requests, a status-code breakdown, average/max latency, cache hit rate, events processed/failed. This is what the frontend's admin dashboard actually fetches and renders as numbers on a page; it's JSON because a React component calling `fetch()` wants JSON, not a Prometheus exposition blob it would have to parse by hand.

Why both exist, instead of just one

They're not redundant — they're answering different questions for different readers:

- `/metrics` (Prometheus format) exists so **Prometheus** can scrape it on an interval, store it as time series, and let Grafana query *trends over time* — rate of requests per second, p95 latency, rate-limit rejection rate over the last hour. That's the whole value proposition of a time-series database: questions shaped like "how has this changed."
- `/api/metrics` (JSON) exists so **a browser tab** can get a cheap, self-contained snapshot — "what's true right now" — without standing up Prometheus/Grafana at all. It's useful specifically when there's no monitoring stack running (e.g. a quick local check), and it's a much smaller, simpler dependency for the frontend than teaching a React component to parse Prometheus's text format or proxy through a Grafana panel.

Two independent recording paths, though — `Metrics.RecordRequest()` (used for the JSON snapshot) and `HTTPRequestsTotal` / `HTTPRequestDuration` (used for Prometheus) are two different

counters tracking the same event from two different call sites, not one feeding the other. They should broadly agree in shape (both count every request), but they're not guaranteed to be byte-for-byte identical — worth knowing if the two ever appear to disagree on an exact number.

What `prometheus.yml` actually scrapes

```yaml
# prometheus.yml
global:
  scrape_interval: 15s
  evaluation_interval: 15s

rule_files:
  - "/etc/prometheus/alerts.yml"

scrape_configs:
  - job_name: 'urlshortener'
    static_configs:
      - targets: ['api:8080']
        labels:
          app: 'urlshortener'
```

One job, one static target: `api:8080` — again the Docker service name, not `localhost`. Every 15 seconds, Prometheus pulls whatever's currently at `http://api:8080/metrics`. The `rule_files` line references `/etc/prometheus/alerts.yml`, which is a path *inside the Prometheus container*, not a file found anywhere in this repo — so either that file is mounted some other way not visible here, or it's a forward-reference to alerting rules that don't exist yet. Worth flagging as `UNKNOWN` rather than assuming either way.

What Grafana actually provisions

Two files under `monitoring/grafana/provisioning/`:

```yaml
# monitoring/grafana/provisioning/datasources/datasource.yml
datasources:
  - name: Prometheus
    type: prometheus
    access: proxy
    url: http://prometheus:9090
    isDefault: true
```

Again, `prometheus:9090` — service name. Grafana doesn't need to be told where Prometheus lives beyond the Docker network address.

```yaml
# monitoring/grafana/provisioning/dashboards/dashboards.yml
providers:
  - name: urlshortener
    type: file
```

```
    options:
      path: /etc/grafana/provisioning/dashboards
```

And the one dashboard that path resolves to, `monitoring/grafana/provisioning/dashboards/urlshortener.json` — title "URL Shortener Overview," four panels, `refresh: "10s"`, default time window `now-1h` to `now`:

1. **HTTP Request Rate** — `sum(rate(urlshortener_http_requests_total[1m])) by (path, status)`
2. **Redirects by Result** — `sum(rate(urlshortener_redirects_total[1m])) by (result)`
3. **Rate Limit Decisions** — `sum(rate(urlshortener_rate_limit_total[1m])) by (result)`
4. **API Up** — `up{job="urlshortener"}`

All four queries map directly onto the metric names shown above — nothing invented, nothing provisioned that the backend doesn't actually emit.

> **★ interview tip Q: Why does the HTTP Request Rate panel use `rate()` instead of just plotting the raw counter?** A: `urlshortener_http_requests_total` only ever goes up — it's a counter. Plotted raw, it's a monotonically climbing line that tells you "how many requests total since the process started," which flattens any interesting signal about *current* traffic into an ever-steepening slope. `rate(...[1m])` converts it into "requests per second, averaged over the trailing minute" — a value that can go up *and* down, which is what you actually want on a dashboard meant to show live traffic shape.

---

23. The observability story, end to end — tracing one real panel

Walking the full path for a single concrete number, rather than describing it abstractly:

1. **A request lands.** Say `GET /abc123` — a redirect lookup. `backend/cmd/api/main.go` has already registered `PrometheusMetricsMiddleware()` ahead of the catch-all `/:shortCode` route (route-ordering matters here exactly as the codebase guide notes — a route registered *after* the catch-all would never reach this middleware at all).
2. **The middleware records it** (`backend/internal/middleware/prometheus.go:11-31`, quoted in full above). It captures `start := time.Now()`, calls `c.Next()` to let the actual redirect handler run, then — once the handler returns — reads the *registered route path* (`/:shortCode`, not `/abc123`), the HTTP method (`GET`), and the final status code (`302`, or `404` if the code doesn't exist). Two things happen in memory inside the API process at this point: `HTTPRequestsTotal.WithLabelValues("GET", "/:shortCode", "302").Inc()` bumps a counter by one, and `HTTPRequestDuration.WithLabelValues("GET", "/:shortCode").Observe(...)` records this request's latency into a histogram bucket. Nothing is sent anywhere yet — this is purely an in-process counter increment.

3. **Prometheus scrapes `/metrics` every 15 seconds** (`prometheus.yml`'s `scrape_interval: 15s`, target `api:8080`). On its next tick, Prometheus makes an HTTP GET to `http://api:8080/metrics`, which the `promhttp` handler inside the Go binary answers by dumping the current in-memory state of every registered metric — including the counter this request just bumped — as plain text. Prometheus parses that text, timestamps it, and appends it to its own time-series storage.
4. **Grafana queries Prometheus.** The "HTTP Request Rate" panel's query — `sum(rate(urlshortener_http_requests_total[1m])) by (path, status)` — isn't evaluated once; Grafana re-runs it on the dashboard's own refresh cycle (`"refresh": "10s"` in `urlshortener.json`). Each time it fires, Grafana sends that PromQL expression to `http://prometheus:9090` (the datasource URL from `datasource.yml`), and Prometheus computes the per-second rate of that counter over the trailing 1-minute window, grouped by `path` and `status`, from the time series it's been accumulating via scrapes.
5. **The panel updates.** The line for `{path="/:shortCode", status="302"}` ticks up, reflecting that this redirect — and every other `/:shortCode` hit in the trailing minute — is now part of the plotted rate. Nothing about this redirect hits a database write on this path (the click event itself goes onto the Redis Stream for the analytics worker, a completely separate flow); the number on this specific panel comes entirely from the in-memory Prometheus counter the middleware bumped in step 2.

That's the full loop, concretely: **app counter → scrape → PromQL aggregation → panel.** No step in that chain touches Postgres, and no step is instantaneous — there's a real, bounded lag (up to the 15s scrape interval, then up to the panel's own 10s refresh) between "a redirect happened" and "the dashboard shows it."

> ★ **interview tip Q: How stale can a Grafana panel be relative to real traffic?** A: Bounded by two independent intervals stacking: up to 15s for Prometheus to next scrape `/metrics` (`prometheus.yml`'s `scrape_interval: 15s`), plus up to 10s for the dashboard's own panel refresh (`urlshortener.json`'s `"refresh": "10s"`). Worst case, a request that just happened can take up to ~25 seconds to visibly move a line on the dashboard. That's a deliberate, reasonable trade for a low-traffic showcase app — it would be the first knob to tighten for anything latency-sensitive.

---

24. The CI/CD pipeline, fully

Two workflow files, two different triggers, chained together deliberately.

`ci.yml` — test, then build/push

Trigger (`.github/workflows/ci.yml:3–7`): `push` to `main`, or any `pull_request` targeting `main`.

`test` **job** (`ci.yml:23–81`), on `ubuntu-latest`:

1. `actions/checkout@v4`.
2. `actions/setup-go@v5`, Go 1.25, with `cache: false` — and that's not an oversight, it's a documented, currently-unresolved debugging decision. The comment (`ci.yml:31–43`) is worth reading in full: every CI run had failed at "Run tests" since the very first commit, yet the identical steps reproduce clean locally on two different architectures — the one variable that *can't* be reproduced locally is GitHub's own module cache, and this repo's `go.mod` has non-standard `replace` directives pointing at GitHub mirror repos instead of the normal module proxy, "exactly the shape that can produce a stale/bad cache entry." Disabling caching was the test of that hypothesis.
3. `go mod download` (`backend/`).
4. **Generate the Prisma client** (`ci.yml:49–70`) — `prefetch` then `generate`. The comment here documents the *actual root cause* that was finally confirmed, quoting the exact failure from a genuinely fresh clone: `internal/db/prisma_store.go:27:10: undefined: PrismaClient`. `internal/db/*_gen.go` is gitignored (per the codebase guide) and only exists after this generation step runs — without it, `internal/db` and anything importing it (`cmd/api`) fail to even *compile*, not merely fail a test. Commit `0ed94c7` is this exact fix, and its message explicitly notes a prior, wrong fix (disabling Go caching, for what turned out to be an unrelated reason) that's "superseded by this commit" — a documented false lead left visible rather than scrubbed.
5. `go test ./... -v` (`ci.yml:72–81`) — no `|| echo` swallow, by explicit design: the comment states plainly that a prior version of this step did swallow failures, which meant "a broken build could still be tagged and pushed to GHCR, and then deployed by deploy.yml." (The codebase guide's own note — "`go test ./...` (currently swallows failure via `|| echo`)" — describes that earlier state; the committed workflow on disk today does not have that fallback.)

`build` **job** (`ci.yml:83–143`), `needs: test` — only runs if tests pass:

1. Checkout.
2. `docker/setup-qemu-action@v3` — cross-architecture emulation, needed because GitHub's hosted runners are amd64-only and the deploy target turned out to be arm64 (see Chapter 25).
3. `docker/setup-buildx-action@v3`.
4. **Smoke-test build** (`ci.yml:105–118`): `push: false`, `load: true`, single-platform (the runner's own amd64) — just confirms `backend/Dockerfile` still builds, loaded into the local daemon for inspection. Deliberately *not* multi-platform, because `load: true` can't accept a multi-arch manifest list.
5. Login to `ghcr.io`, gated `if: github.event_name == 'push' && github.ref == 'refs/heads/main'` — PRs never get this far, only pushes to `main`.
6. **Real push** (`ci.yml:128–143`): `push: true`, `platforms: linux/amd64,linux/arm64`, tagged both `${{ github.sha }}` and `latest`. The `env.IMAGE_NAME` this all resolves under is hardcoded lowercase — `ghcr.io/secretjod/url-shortner` — and the comment at `ci.yml:9–20` explains why it isn't the interpolated `ghcr.io/${{ github.repository }}`: the actual GitHub repo is `secretJod/Url-Shortner`, mixed case, and Docker image references

must be entirely lowercase — the interpolated form produced an invalid reference and silently broke the push step.

`deploy.yml` — triggered by CI's success, not by the push itself

Trigger (`deploy.yml:22-27`): `workflow_run` on workflow `"CI"`, `types: [completed]`, `branches: [main]` — plus manual `workflow_dispatch`. The header comment (`deploy.yml:8-14`) explains the ordering choice precisely: both workflows fire off the same push, but this job — a checkout and a file write — would reach `docker compose pull` within seconds, "long before a hosted runner finishes a Node+Go+Prisma multi-stage build and a GHCR push." Pulling a commit-SHA tag that doesn't exist yet fails with "manifest unknown" and aborts. `workflow_run` makes the dependency on CI's completion explicit instead of racing it.

A `concurrency` group (`deploy.yml:31-33`) limits this to one deploy at a time — two overlapping `compose up` runs would fight over container names and volumes.

The job itself (`runs-on: self-hosted`), gated (`deploy.yml:40-42`) to only actually run when `github.event.workflow_run.conclusion == 'success'` or a manual dispatch — otherwise a `workflow_run` event fires on CI *failure* too, which would deploy a red build.

Checkout pinned to the exact commit CI built (`deploy.yml:45-50`): `ref: ${{ github.event.workflow_run.head_sha || github.sha }}` — because the default ref for a `workflow_run` event is the branch tip, which may have moved since CI started.

Writing `.env` from secrets (`deploy.yml:52-117`) — this is the step worth walking carefully, because the technique is deliberate, not incidental:

```
env:
  POSTGRES_PASSWORD: ${{ secrets.POSTGRES_PASSWORD }}
  REDIS_PASSWORD: ${{ secrets.REDIS_PASSWORD }}
  # ... nine more
run: |
  set -euo pipefail
  required_vars="POSTGRES_PASSWORD REDIS_PASSWORD IP_HASH_SECRET BASE_URL
        CORS_ALLOWED_ORIGINS GF_SECURITY_ADMIN_PASSWORD MAIL_FROM"
  missing=""
  for var_name in $required_vars; do
    if [ -z "${!var_name:-}" ]; then
      missing="$missing $var_name"
    fi
  done
  if [ -n "$missing" ]; then
    echo "::error::Missing required secret(s):$missing — ..."
    exit 1
  fi
  umask 077
```

```
{ echo "ENV=production"; echo "POSTGRES_PASSWORD=${POSTGRES_PASSWORD}"; ... }
      > .env
chmod 600 .env
```

Secrets reach this script via a step-level `env:` mapping — `$VAR`, shell variables — never `${{ secrets.X }}` interpolated directly into the script body. The comment (`deploy.yml:52-63`) states why: a secret value containing `$`, a quote, or a newline must not be able to break the script or leak into the workflow log, which direct `${{ }}` interpolation into a multi-line `run:` block risks. The `"${!var_name:-}"` syntax (`deploy.yml:90-96`) is **indirect expansion** — it reads the value of the variable *named by* `$var_name` (e.g. `POSTGRES_PASSWORD`) without the script's own text ever containing a secret literal, letting one loop check all seven required vars instead of seven repeated `if` blocks each naming a secret directly.

`umask 077` before the write, and `chmod 600 .env` after, mean the file exists without world/group read access from the moment it's created. `ENV=production` is set unconditionally here — this is what turns on `config.go`'s `validateForProduction()` guard, per the codebase guide: without it, that guard never runs and the app would boot happily on committed dev placeholder secrets.

SMTP vars are deliberately *not* in `required_vars` — the comment (`deploy.yml:79-86`) explains that leaving them unset is what selects the MailHog fallback path, which is the actual current operating mode (see Chapter 27).

Pull, then deploy with the mandatory prod overlay (`deploy.yml:119-139`):

```
docker compose -f docker-compose.yml -f docker-compose.prod.yml pull
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --remove-
      orphans
```

Always both files, never the bare compose file — `docker-compose.prod.yml` is what strips host ports from postgres/redis/prometheus/grafana/api, swaps the ghcr.io image in for a local build, disables MailHog's ports, and replaces the `migrate` service with a no-op (see Chapter 25 for why that last one matters on the very first deploy).

Health verification, not just "did `up` exit 0" (`deploy.yml:144-193`): a loop (up to 24 iterations, 5s apart — a two-minute budget) polling `docker compose ps --format '{{.Name}} {{.Health}}'`, failing immediately on any `unhealthy` status, and separately failing if anything is still `starting` after the full budget. Then a second loop (up to 12 iterations, 5s apart) that actually `curl`s `http://localhost/health` **through nginx** — the comment (`deploy.yml:170-175`) is explicit that this is deliberate: "A successful `docker compose up` does not guarantee the app is actually reachable, so hit it for real and fail the job if it doesn't." Both failure branches dump the last 100–200 lines of `docker compose logs` before exiting non-zero, so a failed deploy run leaves a diagnosable trail in the Actions log rather than a bare exit code.

> ★ **interview tip Q: Why `workflow_run` instead of just giving `deploy.yml` the same `push: branches: [main]` trigger as `ci.yml`?** A: Because they'd then be racing each other on the same event, and the deploy job (checkout + write a file + `docker compose pull`) finishes in seconds — long

> before CI's own multi-stage Docker build and GHCR push complete. Pulling a tag that doesn't exist yet fails outright. Making deploy depend on CI's own completion event, rather than mirroring CI's trigger, makes the ordering a structural guarantee instead of a timing coincidence that happens to usually work.

---

25. The real deployment story

Stated plainly, in the order it actually happened — cross-checked against `git log --oneline --all` rather than assumed:

1. The self-hosted runner needed an actual logged-in desktop session, not just SSH access. This is a real operational quirk of GitHub's self-hosted runner on macOS: a runner installed as a `launchd` service under a user context that isn't an active, logged-in GUI session can fail to execute Docker-dependent jobs correctly — Docker Desktop's own daemon socket is only reliably available to processes running inside a real logged-in session, not a bare background/headless one. That's a macOS platform detail, not a Docker Compose or GitHub Actions config problem, and it's exactly the kind of thing that looks like a CI bug until you realize it's an OS session-management quirk. (This detail comes from the operator's own account of standing the runner up; it doesn't appear as commit text in the repo's history, so it's reported here as told, not independently verified against a commit.)

2. The first real deploy attempt failed on an architecture mismatch. Commit `6e67a8f`'s message says this plainly — quoting it directly because it's the clearest first-hand account in the repo:

> deploy.yml's first real run against an Apple Silicon (arm64) machine failed with "no matching manifest for linux/arm64/v8 in the manifest list entries" when pulling ghcr.io/secretjod/url-shortner. Every official base image used here (postgres, redis, nginx, grafana, prometheus, golang) already ships multi-arch — only our own CI-built image didn't, because GitHub's hosted runners are amd64-only and docker/build-push-action defaults to publishing just the runner's own architecture.

The fix was exactly what Chapter 24 already walked through: `setup-qemu-action` plus `platforms: linux/amd64,linux/arm64` on the real push step (`ci.yml:92-140`), merged via PR #6 (`51bc89f`) on the same day, October 15, 2026. This is also the detail that settles the Windows-vs-Mac question raised in the intro: `DEPLOY.md` and the deploy.yml header comment both describe a WSL2/Windows target, but the one commit that actually documents a real failed deploy attempt names an Apple Silicon (arm64) machine explicitly. The repo's own documentation is stale on this point — treat the commit, not the doc, as what happened (`CLAUDE.md` §36).

3. The production overlay deliberately disables automatic schema push, so a brand-new database needed one manual, deliberate `prisma db push` before the stack could come up healthy. This follows directly from how `docker-compose.prod.yml`'s `migrate` service is defined (`docker-compose.prod.yml:91-98`):

```
migrate:
  environment:
    DATABASE_URL: "postgresql://urlshortener:${POSTGRES_PASSWORD:?...}
      @postgres:5432/urlshortener?schema=public"
  command: sh -c "echo 'Skipping automatic schema push in production. Apply schema
      changes deliberately...' && exit 0"
```

In dev, `docker-compose.yml`'s `migrate` service actually runs `prisma db push`, gating `api` via `depends_on: service_completed_successfully` (per the codebase guide). In prod, that command is replaced with a no-op that exits 0 immediately — satisfying `api`'s dependency condition without ever touching the schema. The header comment (`docker-compose.prod.yml:39-55`) explains why: `db push` reconciles the live schema with *no migration history and no rollback path* — fine against throwaway dev data, a real risk of silent data loss against anything that matters. Deliberate consequence: a database that has never seen the schema applied (a fresh Postgres volume, which is exactly what a first deploy to a new machine starts with) will **not** get a schema from `docker compose up` alone in production. That has to be a conscious, separate step — exactly what happened on this deploy: the operator ran `prisma db push` by hand, once, deliberately, against the new database, before the stack reported healthy. This is the overlay doing precisely what its own comments say it's for — trading automatic convenience for a loud, manual gate on anything that touches schema in a real environment.

> **★ interview tip Q: Isn't disabling the automatic migration step in production a regression — doesn't dev get something prod doesn't?** A: It's the opposite of a regression; it's the entire point. `prisma db push` is a *schema-reconciliation* tool with no migration history and no rollback — safe against data you're willing to throw away (dev), unsafe against data you're not (prod, where a column rename or type change could silently destroy rows with nothing to revert to). The overlay doesn't remove schema management from prod — it removes the *automatic, unattended* version of it, replacing it with a conscious, separate, reviewed step. That trade — slower, but loud and deliberate instead of fast and silent — is exactly what you want once there's real data to lose.

---

26. Load testing, for real

`load-tests/README.md` records results from **2026-10-15, against the self-hosted deployment via Cloudflare quick tunnel** — the same day as the multi-arch fix and the deploy workflow's reintroduction, per the git log. Two scripts, both k6.

`rate-limit-validation.js` — confirming the sliding-window limiter

30 rapid requests, one virtual user, `redirects: 0` so k6 sees the raw 302 instead of following it:

```
export const options = {
  scenarios: {
```

```
    burst: { executor: 'per-vu-iterations', vus: 1, iterations: 30, maxDuration:
        '30s' },
  },
};
export default function () {
  const res = http.get(`${TARGET}/${SHORT_CODE}`, { redirects: 0 });
  check(res, { 'is 302 (allowed) or 429 (rate-limited)': (r) => r.status === 302
      || r.status === 429 });
  if (res.status === 429) {
    check(res, { '429 includes Retry-After header': (r) => r.headers['Retry-
        After'] !== undefined });
  }
}
```

The actual recorded result, not rounded: **19 allowed (302), 11 correctly rejected (429)**, every single rejection carrying a `Retry-After` header. That's checked explicitly against the anonymous rate-limit tier the codebase guide documents — "anon 20/min per IP" — and the README states it matches `backend/internal/middleware/ratelimit.go` "exactly." 19 allowed plus 11 rejected out of 30 is consistent with a ~20/min cap under a tight burst (the 20th request's exact pass/fail depends on window boundary timing, which is expected sliding-window behavior, not a bug). 100% of k6 checks passed.

`redirect-latency.js` — legitimate-traffic latency, deliberately under the cap

```
export const options = {
  scenarios: {
    steady: { executor: 'constant-arrival-rate', rate: 10, timeUnit: '1m',
        duration: '1m', preAllocatedVUs: 2 },
  },
  thresholds: { http_req_duration: ['p(95)<500'] },
};
```

10 requests/minute — "well under the 20/min anonymous cap," per the script's own comment — specifically so this run measures real latency instead of triggering rejections. Recorded result: **0% errors, 100% successful redirects, p95 969ms, median 240ms** over 11 requests in 60 seconds.

Worth noting directly: the script's own `thresholds` block asks for `p(95)<500` (under 500ms), and the actual recorded p95 — 969ms — **fails** that threshold. The README doesn't hide this; it states the number plainly (969ms) rather than rounding down toward the threshold or omitting it.

The honest caveat, stated as written

`load-tests/README.md`'s own words, quoted rather than paraphrased:

> the quick-tunnel path (browser → Cloudflare edge → `trycloudflare.com` tunnel → Colima VM → nginx → api) adds real latency that a production network path or a named tunnel would not have. These numbers measure *this* path on this date, not the backend's raw capacity.

That path — Cloudflare's free *quick* tunnel (not a named, stable tunnel), terminating into a Colima VM (a lightweight Linux VM for running Docker on macOS) — has several extra network hops baked into every one of these numbers: the k6-to-Cloudflare-edge leg, the Cloudflare-to-tunnel leg, and the tunnel-to-Colima-VM leg, before a request even reaches nginx. A p95 near a full second is believable for that path and says very little about how fast the redirect handler itself is once a request actually reaches it. This also tells you *why* the p95 misses the script's own 500ms threshold: the threshold was written against application-latency expectations, and what actually got measured includes a free public tunnel's own overhead on top of that.

> **★ interview tip Q: The latency test's own threshold says p95<500ms, and the real result was 969ms. Doesn't that mean the test failed?** A: By k6's own pass/fail criterion, yes — that threshold wasn't met. But reading it as "the backend is slow" would be the wrong conclusion. The README's own caveat names the actual culprit: every one of these requests traveled browser → Cloudflare edge → a free quick tunnel → a Colima VM → nginx → the API, and back. That's several extra network hops that have nothing to do with how fast the Go redirect handler itself runs. The honest move here isn't to quietly raise the threshold or omit the number — it's exactly what the README did: report the real 969ms, and say plainly that it measures *this path on this date*, not the backend's raw capacity.

---

27. What's demo vs. real, stated plainly

This chapter exists so none of the above gets mistaken for more than it is. Three specific places where this repo is honest about synthetic or not-yet-permanent pieces, stated directly rather than left implicit:

`scripts/seed-demo-data.sql` is synthetic, and says so in its own header comment, quoted directly:

> DEMO / LOCAL-ONLY SEED DATA. NOT FOR PRODUCTION. … Every row this script inserts is clearly synthetic … This does NOT simulate, impersonate, or claim to represent actual production traffic or real end users. There is no production deployment of this project serving real users today.

Concretely: 8 short links pointing at real public pages (github.com, wikipedia.org, docs.python.org, etc. — real destinations, to make a demo *look* plausible), with click counts, referrers, countries, and device types all script-generated, not observed. `scripts/README.md` adds the verification detail: this was tested against hand-built tables mirroring `schema.prisma`, not an actual `prisma db push`'d

database — “First run: executed cleanly, inserted 8 links and 2,709 click events,” idempotent on a second run — but with an explicitly named residual unknown, that Prisma’s real generated DDL might differ in some way this script doesn’t account for. It’s also not wired into anything automatic — “not referenced by `docker-compose.yml`, CI, or the deploy workflow.”

MailHog stands in for a real SMTP relay in the current operating mode — an admin relays verification links by hand. `docker-compose.prod.yml`’s own comment (`docker-compose.prod.yml:66-87`) is explicit that this is a deliberate reversal of the usual rule (“never run a dev mail catcher in prod”), made with the trade-off understood: no mail is ever delivered externally; an admin reads the magic-link email out of MailHog’s inbox over an SSH tunnel (`ssh -L 8025:localhost:8025 <user>@<host>`) and passes the code to the user by hand. That “does not scale” (the comment’s own words), and it’s not meant to — it needs no external mail provider, which keeps the whole stack at the project’s ₹0/$0 constraint. The one real security control that makes this acceptable at all is that MailHog’s web UI is never published (`ports: !reset []`) — if it were, anyone reaching the host could read every user’s verification link and claim any account. Setting real `SMTP_HOST` / `SMTP_PORT` / `SMTP_USER` / `SMTP_PASSWORD` secrets switches `internal/mail/mail.go` to an authenticated STARTTLS relay with no code change — the capability exists, it’s just not the currently exercised path.

The Cloudflare quick-tunnel’s URL is not permanent. A `trycloudflare.com` quick tunnel (`cloudflared tunnel --url http://localhost:80`, per `DEPLOY.md`’s notes) mints a fresh, random subdomain every time it starts — it is not a stable or reserved address. That’s exactly why `load-tests/README.md` frames its own results as measuring “this path on this date,” not a durable endpoint: the URL the load tests hit that day will not be the URL a future run hits, and there is no standing guarantee the tunnel (or the machine behind it) is up at any given moment someone might try to reach it. `DEPLOY.md` itself recommends a *named* tunnel (`cloudflared tunnel run`) as the more durable alternative, and is explicit that the quick-tunnel approach is for exactly this kind of demonstration-grade use, not a permanent address.

The throughline across all three: this project is honest, in its own comments and READMEs, about exactly where the line sits between “real mechanism, demonstrated” and “convenience that makes the demo possible.” None of the above should be read as this project claiming a production deployment serving real users today — it isn’t one, and it says so itself.

> **★ interview tip Q: If none of this is “real” production traffic, what’s actually being demonstrated here?** A: A genuinely working pipeline and a genuinely working application, exercised end to end — CI testing and building a real multi-arch image, a real self-hosted runner pulling and deploying it, a real nginx+API+ Postgres+Redis+Prometheus+Grafana stack coming up healthy, a real rate limiter rejecting real excess requests, real latency numbers off a real (if tunneled) network path. What’s *not* real is the traffic volume and the SMTP delivery — there’s no production user base and no outbound mail provider yet. Those are two specific, named gaps, not a blanket caveat over the whole project — and the repo’s own comments draw that line explicitly rather than leaving a reader to guess which parts to trust.